

**FACULTY
OF MATHEMATICS
AND PHYSICS**
Charles University

BACHELOR THESIS

Patrik Backo

**Generating Drum Kit Samples using
Machine Learning**

Institute of Formal and Applied Linguistics

Supervisor of the bachelor thesis: Mgr. Jan Hajič, Ph.D.

Study programme: Computer Science

Prague 2024

I declare that I carried out this bachelor thesis on my own, and only with the cited sources, literature and other professional sources. I understand that my work relates to the rights and obligations under the Act No. 121/2000 Sb., the Copyright Act, as amended, in particular the fact that the Charles University has the right to conclude a license agreement on the use of this work as a school work pursuant to Section 60 subsection 1 of the Copyright Act.

In date

Author's signature

I would like to thank my supervisor, Mgr. Jan Hajič, Ph.D., because without his patience, guidance, and expertise in music and machine learning, this thesis probably would not exist. Additionally, I would like to thank my girlfriend and my family for their endless support.

Title: Generating Drum Kit Samples using Machine Learning

Author: Patrik Backo

Department: Institute of Formal and Applied Linguistics

Supervisor: Mgr. Jan Hajič, Ph.D., Institute of Formal and Applied Linguistics

Abstract: In this thesis, we designed an interactive generative tool based on the Variational Autoencoder (VAE) to synthesise new and interesting drum one-shot samples for electronic music production. We researched audio representations used in audio generation tasks and selected two that suited our settings the best. Furthermore, we created our own dataset of almost 16,000 freely available samples organised into 9 drum categories. Through a series of experiments, we were able to achieve a model that was reconstructing and generating quality samples; however, they contained a specific noise artefact we could not get rid of. Based on the results of PCA and convex combinations methods we found out that the latent space has “meaningful” properties.

Keywords: machine learning, music generation, electronic music

Název práce: Generování vzorků bicích souprav pomocí strojového učení

Autor: Patrik Backo

Katedra: Ústav formální a aplikované lingvistiky

Vedoucí bakalářské práce: Mgr. Jan Hajič, Ph.D., Ústav formální a aplikované lingvistiky

Abstrakt: V této práci jsme navrhli nástroj pro interaktivní generování vzorků pro jednotlivé údery bicích nástrojů (one-shot samples) pomocí variačních autoenkodérů (VAE), v kontextu produkce elektronické hudby. Prozkoumali jsme reprezentace používané pro úlohu generování zvuku a vybrali dvě nejlépe vyhovující. Dále jsme sesbírali vlastní datovou sadu téměř 16,000 volně dostupných vzorků, které jsme setřídili do 9 kategorií bicích nástrojů. Sérií pokusů jsme pak došli k modelu, který rekonstruuje a generuje vzorky s postačující kvalitou, ačkoliv stále obsahují jeden šumový artefakt. Na základě pokusů s PCA a konvexními kombinacemi jsme zjistili, že natrénovaný latentní prostor má interpretovatelnou a tedy ovladatelnou topologii..

Klíčová slova: strojové učení, generování hudby, elektronická hudba

Contents

1	Introduction	7
1.1	Thesis overview	7
2	Related Work	9
2.1	Audio Synthesis	9
2.2	Variational Auto-Encoders	10
2.3	Generative adversarial networks	11
2.4	“Honorable Mentions”	13
3	Methods	15
3.1	Variational autoencoders	15
3.1.1	Generative models	15
3.1.2	Autoencoders	15
3.1.3	VAE Introduction	17
3.1.4	Loss Function	17
3.1.5	Training	18
3.1.6	Advantages and Disadvantages of VAE	20
3.2	Data representations and processing	21
3.2.1	Audio signal	21
3.2.2	Spectrograms	21
3.2.3	STFT	23
3.2.4	MFCC	25
3.2.5	Griffin-Lim algorithm	26
3.3	Encoder and Decoder Architecture	28
4	Dataset	30
4.1	Organization	30
4.2	Where samples come from?	31
4.3	Data gathering	32
4.4	Data Cleaning	32
5	Results examination	35
5.1	Automatic examination	35
5.1.1	Loss functions	35
5.1.2	Distance Histogram	35
5.1.3	PCA	36
5.1.4	Fréchet Audio distance	37
5.2	Human examination	37
5.2.1	Reconstruction	37
5.2.2	Sampling from the latent space	38
5.2.3	PCA shift	38
5.2.4	Convex combinations	39

6	Architectures, Experiments and Results	40
6.1	Architectures	40
6.1.1	VAE_3	41
6.1.2	VAE_4	41
6.1.3	VAE_magpie_1	42
6.2	Experiments	46
6.2.1	MFCC	46
6.2.2	VAE_3 experiments	47
6.2.3	STFT	48
6.2.4	VAE_4 experiments	49
6.2.5	VAE_magpie_1 experiments	50
6.3	Failed Experiments	51
7	Conclusion	54
7.1	Future work	55
	Bibliography	56
	List of Figures	59
	List of Abbreviations	60
A	Attachments	61
A.1	Installation instructions	61

1 Introduction

Music has played a significant role in human lives for centuries, and with the rise of technologies and computers, it has evolved into new directions, one of which is electronic music. Electronic music dates back to the 1960s and 1970s, when the first synthesizers became available. A synthesiser is a machine that allows the artist to electronically synthesise sounds or use real recorded sounds and modify them with various electronic effects.

Today, artists usually use music production software, rather than hardware, to create their songs, also known as “tracks” in the world of electronic music. They connect short recordings of various sounds, known as samples or one-shots, into loops and melodies, which then form tracks. These samples can be basically anything: sounds of musical instruments, vocals, quotes from movies, or artificial sounds made by the synthesizers.

We came up with the idea of developing a machine learning tool that could simplify or enhance the process of producing electronic music. After a few meetings with artists who produce electronic music, we discovered that finding the right samples that correspond to the artist’s ideas is a significant part of producing a track. As a result, artists either own vast libraries containing thousands of samples or can access even more extensive libraries online. The artist then has to undergo a quite long and boring process of rummaging through these libraries and listening to samples until he finds the right ones.

Therefore, we came up with the topic for this thesis, which is to create an interactive generative model that would be able to generate new samples based on some input or prompt and allow the artist to combine the sounds or slightly modify them with buttons and knobs that do intuitively meaningful but not necessarily pre-defined things. We chose to focus our experiments on one-shots, which are elementary units from which tracks are constructed. And more specifically on drum one-shots because drums are fundamental to tracks in nearly all electronic genres.

In short, we will try to design a drum machine, or more accurately a drum sound synthesizer, based on deep learning methods. We aim to utilise the potential of deep learning in an artistically interesting and enriching way by enabling the synthesis of sounds that are not necessarily associated with the usual categories of physical percussion instruments.

The contributions of this thesis are: a dataset containing nearly 16 samples categorised into 9 drum-type categories, a generative model that shares a latent space with samples from all drum categories, and an examination pipeline that generates samples using a variety of latent space sampling techniques.

1.1 Thesis overview

In the second chapter, “Related Work” 2, we will discuss similar research that has been done in this field. We will begin with studies in audio synthesis employing statistical techniques, proceed to the generation of speech and sounds, and finish with articles that focus on the development of a deep learning drum machine. We will also mention some intriguing papers that share our goal of

creating an electronic music production tool.

In the third chapter, “Methods” 3, we will explain our architecture of choice, the Variational Autoencoder, and introduce several representations of audio signals that are used for audio generation, especially the 2D time-frequency representation called a spectrogram. Additionally, we will explain in detail the spectrogram representations we used.

The fourth chapter, “Dataset” 4, will introduce the dataset that we developed specifically for our topic and explain what steps we took in order to create it. We will describe the processes of collecting, cleaning, examining, and organising the drum one-shots.

In the fifth chapter, “Results Examination” 5, we will explain how we examined the results of a generative model for audio samples and introduce methods for generating drum one-shots with the variational autoencoder architecture.

In the sixth chapter, “Experiments and Results” 6, we will describe the exact VAE networks we designed and present successful experiments that we conducted, as well as the parameters of the network and the spectrogram representations that were used in these experiments. At the end, we will also mention experiments that failed.

The seventh and last chapter, “Conclusion” 7, will summarise all the work and experiments conducted in this thesis and outline potential directions for future research. ¹

¹In this thesis we used several LLM tools in accordance with the MFF UK Code of Ethics Article 8 only in a complementary role, as follows: “Chat-GPT” for generating an overview about some method or technology as a starting point in ongoing research (None of the generated text was used in the thesis); “Github Copilot” for coding assistance, for example, generating of code documentation or type hints; and “QuillBot” for grammar checking.

2 Related Work

The generation or synthesis of drum audio with machine learning for use in electronic music production is a task that has been studied from a few different perspectives.

First of all, it is essential to define the musical term “timbre”, as it is crucial for understanding the following papers. Timbre, also known as tone colour or tone quality, is a sound characteristic that allows us to distinguish between different sources producing the same pitch and volume. For example, it enables us to tell apart different musical instruments or human voices, even if they are playing or singing the same note at the same loudness ¹.

When applying machine learning to data, we have to decide what representation of the data we will use. The initial experiments in audio synthesis and generation focused on waveform representation without any preprocessing due to the scarcity of alternative representations with suitable inverse functions. However, the development of new representations and methods led to research into 2D representations of a wave, known as spectrograms. Chapter “Methods” 3 provides more information on spectrograms.

2.1 Audio Synthesis

Hawley et al. [1] present a comprehensive analysis of two methodologies employed for audio synthesis. The first approach involves physics-based modelling that aims to capture and simulate various characteristics of musical instrument play. An interesting outcome of this approach is a machine that simulates a clarinet’s play. The machine applies forces and air pressure to the clarinet mouthpiece, just as a musician does while playing on a clarinet. The second method leverages machine learning and data, which has gained traction due to the increasing focus on research in machine learning and, more specifically, deep learning.

The initial attempts primarily focus on human speech rather than just music. In their work, Mehri et al. [2] introduces SampleRNN, a model based on recurrent neural network architecture. The model’s input and output are a raw waveform of audio, and it was trained and tested on three different datasets separately containing hours of female speech, human vocal sounds like screaming or coughing, and piano songs.

Oord et al. [3] investigated another approach where he and other authors use an autoregressive model with 1D convolutional layers, and as in the previous model, the input is a raw wave. Autoregressive means that a wave is predicted as a sequence, where every other value is conditioned on all the previous ones. The results of their experiments indicate better performance than SampleRNN, with much faster training time thanks to the better parallelization of this architecture. This makes sense in speech synthesis, but not so much if we want to generate new music sounds.

This issue is addressed by Engel et al. [4]. Strongly influenced by WaveNet, they modify it by adding an autoencoder, therefore making it more suitable for

¹For additional details, visit <https://en.wikipedia.org/wiki/Timbre>

musical note synthesis. The autoencoder part provides a latent space from which we can sample a vector that will influence what the decoder generates, eliminating the need for external conditioning. Apart from this, they introduced NSynth, a dataset of 300,000 one-shots from 1000 musical instruments (unfortunately no drums) with various pitches used in the training of the model.

2.2 Variational Auto-Encoders

This section is devoted to papers that use the variational autoencoder, abbreviated as VAE, introduced in Kingma and Welling [5]. This architecture will be explained in a later chapter, “Methods” 3, because it is the architecture used in this thesis.

Several studies introduced the concept of a timbre space, in which every sound is a point and its coordinates provide values for its timbral properties. These studies attempt to create timbre spaces using human ratings organised by multi-dimensional scaling, but there are limitations to these methods, mainly that their spaces are discrete and non-expandable, so with every new sound, the space has to be recomputed.

Esling et al. [6], affected by WaveNet autoencoders, had an idea of using Variational Auto-Encoder containing convolutional layers together with the NSynth dataset to create a timbre space that would be continuous and could generalise to new tones. They use VAE conditioned by human ratings to create a continuous latent space that functions almost like a theoretical timbre space. The network can explain timbral properties and the relationship of tones encoded into latent space, or it is possible to navigate through the latent space and generate sounds with desired timbral properties.

Another way to look at timbre is through timbre transfer, which means the ability to modify a tone with a certain timbre and pitch so it preserves its pitch but changes the timbre. For example, we convert a piano C tone to a violin C tone. The research was focused on one-to-one or one-to-many models, which means having one input timbre and one or multiple output ones. Bitton et al. [7] introduce a many-to-many model using a modified VAE architecture that they call MoVE (Modulated Variational Auto-Encoders). They report a successful transfer model as well as an option to synthesise a sound from a reduced set of control parameters while remaining lightweight.

Aouameur et al. [8] are trying to achieve very similar results with similar intentions as we do. Their model is based on architecture, which is a variation of a VAE called *CWAE* (*Conditional Wasserstein Autoencoder*), introduced by Tolstikhin et al. [9]. They also introduced a dataset of 40,000 samples from 11 drum categories. Their work consists of three parts:

1. CWAE trained on the dataset to create a latent space, from which they could sample and decode it into a new sample. The encoder and decoder of CWAE are based on 2D convolutional layers.
2. Inverse spectrogram model. The authors decided to use a mel-spectrogram representation for their input waves, but a precise inverse mel-spectrogram function does not exist, so they had to create another neural network that was trained to recreate a wave from a spectrogram.

3. The Max4Live interface is designed for users to interact with the model, or more concretely with the latent space, decoder, and inverse spectrogram model, because during generation the encoder is no longer used. Additionally, their interface allows direct integration into Ableton Live, one of the most popular electronic production programmes.

Four options govern the generation from the latent space: the first option determines the category of the drum to generate, while the other options manipulate knobs to shift the vector in the latent space towards one of the three most influential dimensions produced by PCA.

2.3 Generative adversarial networks

The most potent research direction seems to be GANs. Generative adversarial networks (Gans) are a class of deep learning models designed for generative tasks, introduced by Goodfellow et al. [10]. GANs consist of two neural networks: a generator and a discriminator, which are trained simultaneously through adversarial processes. The generator network aims to produce synthetic data samples that mimic the real data distribution, while the discriminator network evaluates these samples to distinguish between real and generated data. The training process involves a minimax game, where the generator seeks to minimise the discriminator’s ability to classify generated samples correctly and the discriminator strives to maximise its classification accuracy. This dynamic results in the generator progressively improving its outputs, ultimately generating data that is nearly indistinguishable from real data.

One of the first GAN-based models for audio synthesis and generation WaveGAN and SpecGAN were introduced by Donahue et al. [11]. They studied this architecture on similar tasks as the first WaveNet [3], human speech, bird sounds, and instrument sounds. Another parallel with WaveNet is the usage of raw wave representation, due to the lack of precise inverse functions for spectrogram representations. However, they decided to investigate a spectrogram representation too, so together with WaveGAN, they produced a spectrogram model, SpecGAN. A magnitude of STFT (short-term Fourier transform) with additional preprocessing serves as the training spectrogram. During generation, they generate magnitude and estimate phase with an iterative Griffin-Lim algorithm [12].

Furthermore, they created a drum machine based on WaveGAN and presented a nice demo of it. However, since drum sample generation was not their primary goal, they did not provide any details on how they trained this model or what data they used. The results show comparable results to WaveNet, with a significant decrease in generation time, and they also suggest that WaveGAN performed better than SpecGAN.

Marafioti et al. [13] want to show that the time-frequency representation of sound waves, spectrograms, can indeed work for sound synthesis when combined with GANs; therefore, they introduce TiFGAN. In their work, they use STFT, thoroughly explain the theory behind it, examine multiple phase estimation methods, and test STFT parameters to find the best settings. They trained and examined the model on a subset of data that WaveGAN had used: human speech

and instrumental sounds. They provide results that indicate better performance than previous GAN audio synthesisers, WaveGAN and SpecGAN.

Engel et al. [14] follow up their previous work on WaveNet autoencoders, [4] and introduce a model trained on the NSynth dataset called GANSynth. GANSynth outperforms its predecessor, allows unconditional generation, and is approximately 54000 times faster. Another difference is that GANSynth uses spectrogram representation instead of raw waves. Similar to SpecGAN, GANSynth bases its spectrogram representation on STFT, but employs a distinct phase retrieval process that involves estimating IF (instantaneous angular frequency) spectra through phase unwrapping.

In the next research, Nistal et al. [15], Drysdale et al. [16], [17], Lavault et al. [18], focus on simulating a drum machine with GAN models. This entails developing a model capable of producing drum samples, along with control switches and knobs that can influence the generation process.

Nistal et al. [15] introduce DrumGAN, a network that generates drum samples with high-level timbral features. These features are designed based on a study of popular timbre ratings given to a collection of sounds obtained from a popular music production site. These features are brightness, hardness, depth, roughness, boominess, warmth, and sharpness, ranging from 0 to 100. The generator’s input is a randomly sampled vector from latent space, along with the values of timbre features. In addition to the normal discriminator output, the model also predicts the values of the timbral features. The authors decided to use STFT spectrogram representation, the model is trained on both real and imaginary components (magnitude and phase), enabling them to invert the spectrogram with an inverse Fourier transform without the need to estimate phase. Lastly, they trained their model on a private dataset consisting of 300,000 drum samples, padding or cutting them to 1 second of audio at 16 kHz.

Drysdale et al. [16] propose the idea of using conditional Wasserstein GAN [19] for drum synthesis. A seasoned electronic music producer’s libraries may contain thousands of samples, but this is still a relatively small amount of data compared to previous music synthesis work, which often used datasets with tens of thousands or even hundreds of thousands to train the models. Therefore, the authors aim to develop a less data-hungry model that any producer could easily train or fine-tune with samples from his libraries. For that reason, they present a private dataset with 9000 drum samples to simulate a producer’s library. The same authors follow their work in [17], where they extend the previous work with another network, an encoder that is trained to encode samples into the latent space of the GAN, it is called the GAN inversion. This allows the user to choose their own sample, encode it into latent space, adjust it, and then reconstruct it into a wave using the generator. Another extension is control over the latent space with PCA’s most influential dimensions, as in some of the previous papers. Hence, they are trying to train unsupervised control over a sound’s timbre.

Lavault et al. [18] studied the behaviour of StyleGAN [20] for audio synthesis and developed the StyleWaveGan model. StyleGAN introduces several features that provide more stable training and better generation control. The model is trained on a subset of the publicly available dataset ENST-Drums, which contains recordings of several drummers playing one-shots, loops, and drum solos. They only use one-shots, whose number is around 350, which is too low for quality training,

so they use several data augmentation methods: a gain applied to transient/attack components, noise components, as well as independent transposition of the signal source and the spectral envelope.

2.4 “Honorable Mentions”

This section lists papers that do not study sound synthesis or generation and therefore could not be included earlier. However, they present captivating concepts closely linked to electronic music production and machine learning.

Ramires et al. [21] focus on a very similar task as DrumGAN [15], which is drum sample generation with high-level timbral features using the Wave-U-Net architecture introduced by Stoller et al. [22]. A U-net architecture for segmentation tasks in medicine was proposed by Ronneberger et al. [23]. The network receives an input image, downsamples it several times with convolution layers, and then upsamples it back with transposed convolution layers into the shape of the original image. Additionally, on each upsampling level, there is a skip connection from the downsampling layer of the same shape. As a result, the architecture is shaped like a U, hence its name. The wave-U-Net modifies this architecture to have 1D input and 1D convolution layers, so it can process raw waves. In the original paper, the Wave-U-Net is developed for audio source separation, a task where the input is a music recording and the output is the separated sound of an instrument specified before the separation. Ramires et al. [21] input the Wave-U-Net with timbral features and an envelope, a feature that describes the energy of the sounds over time and is normally available to users in drum synthesisers, and on output expect a sample with specified timbral features. They utilise two datasets: one consists of 10,000 percussive samples, while the second one originates from the author’s libraries.

Another challenge in electronic music production is creating a drum loop from one-shot samples that plays for the majority of a track. Alain et al. [24] introduce the DeepDrummer, a neural network influenced by GANs that can generate drum loops and learn from human feedback interactively. The model consists of two parts: the generator and the critic. Drum loops are produced by creating a grid of zeros and ones, where every row of a grid represents a drum one-shot and column time. The grid is then looped from left to right at the tempo the artist chooses, and every time there is a one in a column that is currently played, the sound connected to the row where number one is located will play. This is simulated by DeepDrummer, it has a generator that is a Metropolis-Hastings algorithm that randomly generates the grid and chooses four drum one-shots for every row of a grid. Then the loop is fed to the critic, and a neural network inspired by GAN’s discriminator will decide whether the user will like the loop or not.

There are two phases.

1. Phase 1, where the generator generates a loop, the user will label it with like or dislike, and then the critic learns from this feedback. This process is repeated 80 times, during which the critic is trained to understand what loops the user might like.
2. Phase 2, where the generator generates loop after loop until the critic says the user will like it, and after that, it is played to the user.

Results of an experiment done on 25 human participants indicate that before training, the average like/dislike ratio was about 37% whereas after training the ratio increased to 54%.

3 Methods

Our goal in this chapter is to present and explain the techniques we used to create an interactive drum sample generator for our thesis.

Two main sections will be presented. In the first, we will describe the architecture of choice, called the Variational Autoencoder. In the second section, we will explain the basic properties of a digital audio signal and the two main directions of representing an audio signal: the waveform and the spectrogram. Finally, we will explain the two spectrogram representations we selected for this thesis: MFCC and STFT.

3.1 Variational autoencoders

The architecture we selected for our experiments is a generative network architecture called *Variational Autoencoder*, abbreviated as *VAE*, introduced by Kingma and Welling [5]. In the next few subsections, we will give a brief introduction to this architecture and to generative models in general.

3.1.1 Generative models

The aim of a generative model is to estimate $\mathbf{P}(\mathbf{X})$, the distribution of a random variable $\mathbf{X}$, given a set of observations of this variable. Usually, we would like to be able to sample from this distribution. In our case, a training dataset of drum samples serves as a collection of realisations, and we want to estimate a data-generating distribution from which we can generate new samples or samples from our dataset.

One of the possible approaches to estimating the distribution $\mathbf{P}(\mathbf{X})$ is to make the assumption that random variable $\mathbf{X}$ depends on latent variable $\mathbf{z}$:

$$P(X) = \int_{z \in Z} P(z)P(X|z) = \mathbb{E}_{z \sim P(z)}[P(X|z)]$$

We can think of $\mathbf{z}$ as some description of $\mathbf{x}$ we want to generate, although in practice it is a vector of real values.

The conditional probability $P(X|z)$ will be estimated by a neural network called a decoder, and we will denote this estimation as $P_\theta(X|z)$.

3.1.2 Autoencoders

One useful way to obtain a “description” $\mathbf{z}$ for our $\mathbf{x}$ is an architecture called the *Autoencoder* shown in figure 3.1.

Autoencoder, abbreviated as *AE*, uses two neural networks, Encoder and Decoder, to project some datum $\mathbf{x}$ onto a vector $\mathbf{z}$ located in latent space, and then uses Decoder to decode the $\mathbf{z}$ back to $\mathbf{x}$. The decoder provides us with the conditional probability $P_\theta(X|z)$.

The AEs’ trick lies in the fact that the latent space dimensionality is significantly smaller than the input dimensionality, which forces the networks to learn

non-trivial input features during the input compression process that retain as much information about the data distribution as possible.

AE also offers a second usage option: adding noise to the input but using the original unnoised input when computing the loss function. Again, the networks must acquire associations from the inputs to eliminate the noise, a process known as denoising. Both the bottleneck in the latent space and the denoising process make sure that the autoencoder does not just memorize the inputs.

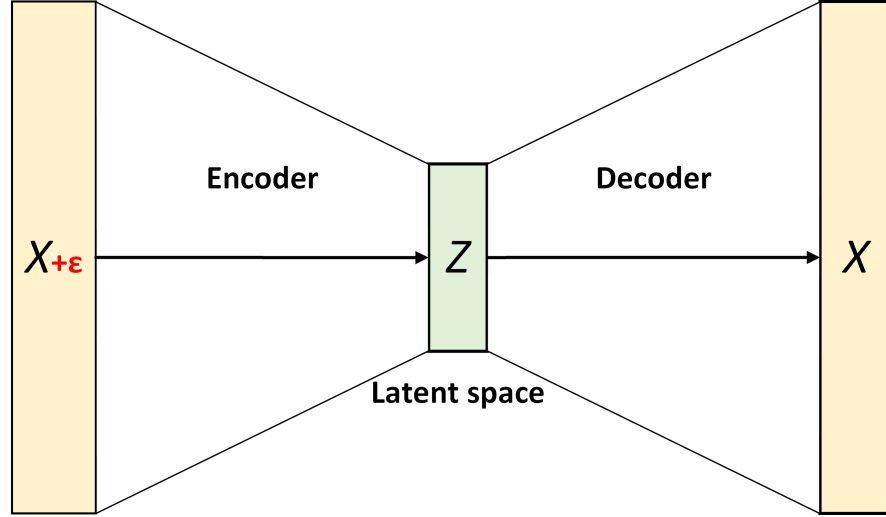


Figure 3.1 autoencoder

Unfortunately, the AE will fail to generate new data. We would like to sample a random point from the latent space and then decode it with the decoder into a datum that is somehow similar to our training data but not the same. However, the probability of generating anything reasonable with this approach is low because the loss of the AE only requires correct reconstruction of training data and does not enforce meaningful representation for all the latent space points, resulting in a discrete latent space. This issue is illustrated in figure 3.2.

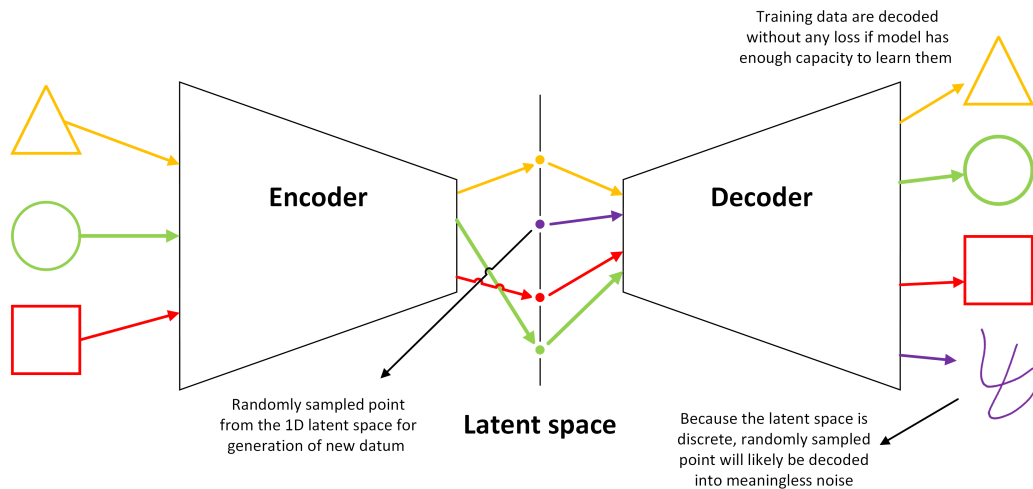


Figure 3.2 Issue with autoencoders generating new data.

3.1.3 VAE Introduction

Even though AE has a problem generating new data, it is a useful starting point for our generative model. Kingma and Welling [5] studied this problem and introduced a solution called Variational autoencoders, abbreviated as VAE. The main idea is that instead of encoding an input onto a single point in latent space, it will encode it onto a distribution of points. Additionally, VAE will have an adjusted loss function that enforces certain distribution parameters.

Ideally, we aim to cover the entire latent space, or at least a limited portion of it, with distributions encoded from the training data. As a result, we could randomly sample from the latent space, decode it, and receive relevant output. This goal is illustrated in figure 3.3.

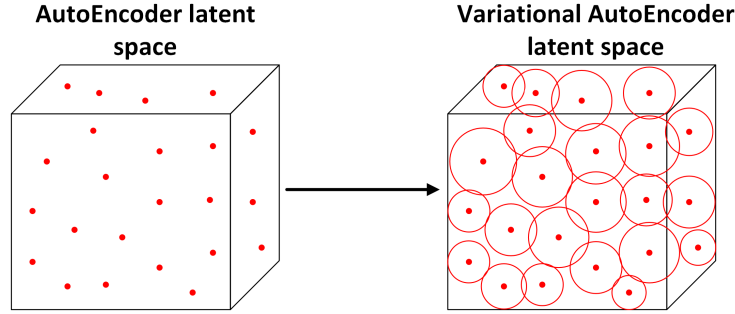


Figure 3.3 Latent space of autoencoder and Variational autoencoder

Now we will explain more formally how to get from AE to VAE. We assume that the prior distribution $P(Z)$ is fixed and independent on X , which means we have a fixed distribution from which we will randomly sample points z .

We will approximate $P(x|z)$ using the decoder network - $P_\theta(x|z)$. For training an AE, we first need to find a distribution of z if we already know x , which means $P(z|x)$. We could estimate it by posterior distribution $P_\theta(z|x)$ except this posterior is usually unsolvable; therefore, we will estimate $P_\theta(z|x)$ with trainable $Q_\varphi(z|x)$ represented by another network encoder.

3.1.4 Loss Function

As usual, we will be maximising log likelihood: $\log P_\theta(x)$, but as we mentioned in subsection 3.1.1, it is not obvious how to compute the likelihood $P_\theta(x)$, so we will express it with latent variable z .

$$\log P_\theta(x) = \mathbb{E}_{z \sim P(z)} P(X|z)$$

We we approximate the expected value using a single sample from prior $P(Z)$, it would be an unbiased estimation; however with an enormous variance; therefore, for most z the probability $P_\theta(x)$ will be very close to zero.

However, we still have an encoder that can generate a distribution of z given x :

$$\begin{aligned} \log(P_\theta(x)) &= \log(\mathbb{E}_{z \sim P(z)} [P(x|z)]) \\ &= \log\left(\int_{z \in Z} P(z) \cdot P(x|z)\right) \end{aligned}$$

$$\begin{aligned}
&= \log\left(\int_{z \in Z} P(z) \cdot P(x|z) \cdot \frac{Q_\varphi(z|x)}{Q_\varphi(z|x)}\right) \\
&= \log\left(\int_{z \in Z} Q_\varphi(z|x) \cdot P(x|z) \cdot \frac{P(z)}{Q_\varphi(z|x)}\right) \\
&= \log(\mathbb{E}_{z \sim Q_\varphi(z|x)}[P(x|z) \cdot \frac{P(z)}{Q_\varphi(z|x)}])
\end{aligned}$$

Now we can use Jensen's Inequality for the expected value $\mathbb{E}[g(X)] \leq g(\mathbb{E}[X])$ for $g(X)$ concave function. Because $\log$ is a concave function:

$$\begin{aligned}
&\log(\mathbb{E}_{z \sim Q_\varphi(z|x)}[P(x|z) \cdot \frac{P(z)}{Q_\varphi(z|x)}]) \\
&\geq \mathbb{E}_{z \sim Q_\varphi(z|x)}[\log(P(x|z) \cdot \frac{P(z)}{Q_\varphi(z|x)})] \\
&= \mathbb{E}_{z \sim Q_\varphi(z|x)}[\log(P(x|z)) + \log(\frac{P(z)}{Q_\varphi(z|x)})] \\
&= \mathbb{E}_{z \sim Q_\varphi(z|x)}[\log(P(x|z))] - \mathbb{E}_{z \sim Q_\varphi(z|x)}[\log(\frac{Q_\varphi(z|x)}{P(z)})]
\end{aligned}$$

The second expected value is the formula for KL divergence, so we can write:

$$= \mathbb{E}_{z \sim Q_\varphi(z|x)}[\log(P(x|z))] - D_{KL}(Q_\varphi(z|x) || P(z))$$

The resulting formula is called the evidence lower bound (ELBO) and is a lower bound to log-likelihood. Therefore, when we are maximising ELBO, we know that log-likelihood is at least as large. We will denote the ELBO loss as:

$$\mathcal{L}(\theta, \varphi; x) = \mathbb{E}_{z \sim Q_\varphi(z|x)}[\log(P(x|z))] - D_{KL}(Q_\varphi(z|x) || P(z))$$

3.1.5 Training

Now that we have an idea of what we would like to achieve with the VAE and we have derived the loss function, we can put all the pieces together, work out the remaining ones, and make the architecture ready for training and usage.

In machine learning, it is typical to minimise the loss function, so instead of maximising $\mathcal{L}(\theta, \varphi; x)$ we will minimise $-\mathcal{L}(\theta, \varphi; x)$.

$$-\mathcal{L}(\theta, \varphi; x) = \mathbb{E}_{z \sim Q_\varphi(z|x)}[-\log(P(x|z))] + D_{KL}(Q_\varphi(z|x) || P(z))$$

$\mathbb{E}_{z \sim Q_\varphi(z|x)}$ will be estimated by a single sample from $Q_\varphi(z|x)$, so we encode x with the encoder that will give us a whole distribution $Q_\varphi(z|x)$ and we will sample a single z and estimate the expected value with it.

The distribution $Q_\varphi(z|x)$ will be parameterized as a normal distribution $\mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\sigma}^2)$ and prior ¹ $P(z) = \mathcal{N}(\mathbf{0}, \mathbf{I})$; therefore, the encoder will be predicting two parameters $\boldsymbol{\mu}, \boldsymbol{\sigma}$ when given input x . However, $\boldsymbol{\sigma}$ has to be always positive, so instead of $\boldsymbol{\sigma}$ we will be predicting $\log(\boldsymbol{\sigma}^2)$ and then using exponential activation

¹Lower cased bold symbols represent vectors, and upper cased bold symbols Matrices. For example $\boldsymbol{\mu}$ is mean vector of the normal distribution. And $\mathbf{I}$ is unit matrix.

$\exp(\log(\sigma^2))$ to get variance σ^2 or $\exp(0.5 \cdot \log(\sigma^2))$ to get std - standard deviation σ . This way, we ensure that the predicted std will always be positive.

This makes sense for several reasons:

- sampling from a normal distribution is efficient.
- KL divergence can be computed analytically.
- With a trick that we will show later, we can backpropagate through it.
- Additionally, according to the maximum entropy principle, the most general distribution with two parameters, mean and variance, is a normal distribution.

Now there is still a problem of how to backpropagate through normal distribution, there is no clear way to differentiate it, but we can use a reparametrization trick. It means when we are sampling z from the normal distribution:

$$z \sim \mathcal{N}(\mu, \sigma^2)$$

can be written as ²

$$z \sim \mu + \sigma \odot \mathcal{N}(\mathbf{0}, \mathbf{I})$$

This formulation allows us to differentiate z with respect to μ and σ ; therefore, updating weights in the network. Figure 3.4 is an illustration of the whole VAE.

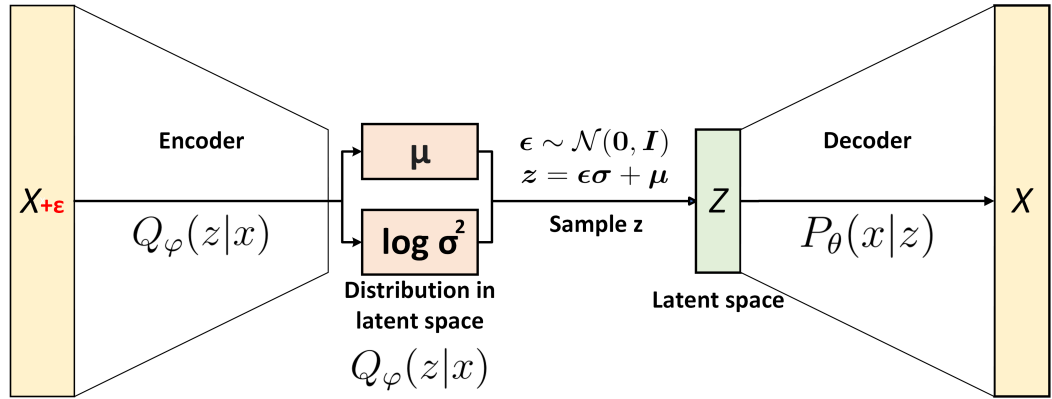


Figure 3.4 Variational autoencoder

Lastly, if we look at the loss function, it consists of two intuitive components:

$$-\mathcal{L}(\theta, \varphi; x) = \mathbb{E}_{z \sim Q_\varphi(z|x)}[-\log(P(x|z))] + D_{KL}(Q_\varphi(z|x) || P(z))$$

- **Reconstruction loss** $\mathbb{E}_{z \sim Q_\varphi(z|x)}[-\log(P(x|z))]$ measures how well the model reconstructs the input. The loss can have different values for the same input due to the estimation of the expected value with a single sampled z . As in AE, noise can be added to the input. For this part, we use MSE, mean square error, as is typical in our setting.

² $\odot$ is element-wise vector or matrix multiplication.

- **Latent loss** $D_{KL}(Q_\varphi(z|x)||P(z))$ over all x measures the difference between $Q_\varphi(z|x)$ distribution generated by encoder and prior $P(z) = \mathcal{N}(\mathbf{0}, \mathbf{I})$ which is independent on x . This part enforces a well-covered latent space around the center. We can compute the latent loss analytically for normally distributed prior and encoder distributions, as we mentioned earlier:

$$-\frac{1}{2} \cdot (1 + \log(\sigma^2) - \mu^2 - \sigma^2)$$

These two loss components are partially at odds with each other.

Reconstruction loss prefers smaller variances, or ideally zero, because then the randomness of sampling is weaker and the model can produce better reconstructions, but this way we are coming back to AE and discrete latent space, which are unsuitable for generating new data.

On the other hand, latent loss is fully satisfied when all the distributions are the same as before, with mean = 0 and variance = 1. However, any point sampled from latent space would be decoded into the same noise made from a mix of training data.

In order to achieve some kind of equilibrium where we can sample points from latent space and get reasonable data, we need regularisation of either latent or reconstruction loss. Figure 3.5 illustrates the whole issue.

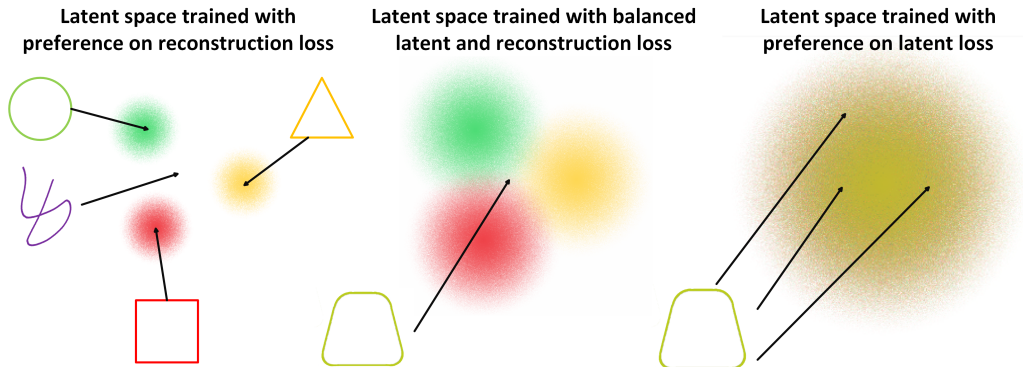


Figure 3.5 Latent spaces trained with various regularisations of loss

3.1.6 Advantages and Disadvantages of VAE

In the last subsection dedicated to VAE, we will discuss its strengths and weaknesses, as well as why we decided to use it for our work.

VAE has straightforward training and has a well-structured latent space in which multiple coordination methods can be used for interactive sampling, which is an advantage over GAN, which usually has disordered latent space. We will discuss these sampling methods in the future section 5.2. Generating is slower than in GAN, but faster than in autoregression models.

The disadvantage is that training tends to be unstable due to the difficulty of finding the right regularisation for loss, so both components are well balanced and meaningful generation can be obtained. The right value for regularisation also differs with every model and data preprocessing technique, so every time we wanted to try something new, we again had to find the right regularisation value.

Even though we found the right regularisation, the generated data from VAE is likely to be a little bit noisy.

All taken into account, we decided to use VAE for its structured latent space and various techniques of navigation through it, because, thanks to it, we can give artists an interactive way to generate new drum one-shots for their tracks.

3.2 Data representations and processing

Processing data for further use in machine or deep learning is a common practice because the suitable learning techniques and also results can change significantly with various preprocessing methods. In comparison with other audio related tasks, especially classification, audio generation and synthesising, has disadvantage, because all the preprocessing methods we use for data has to be invertible in order to get meaningful result out of model's generated output.

There are numerous representations for audio record and we mentioned several that are used in audio synthesis and generation in related work chapter 2. These representations can be divided into two main directions, using raw waveform or a spectrogram, but both of these need the same preceding steps.

3.2.1 Audio signal

The audio signal is captured and stored as a sequence of discrete amplitude samples of the audio wave, sorted by time, in the same way other signals are. They are captured at a certain frequency of samples per second, called sample rate and it is measured in Hertz, (Hz). The sample rate also limits the frequency range we can capture with this approach. A standard sample rate used in music production is 44.1 kHz, which captures frequencies up to 20 kHz, which is the accepted upper limit of the range of human hearing. Lower sample rates are also not unusual.

The sample rate is crucial information about digital audio record, because when interpreted with a wrong sample rate, the record becomes random noise. Therefore, first step in any audio preprocessing pipeline for audio generation should be selecting a uniform sample rate, checking whether all of the samples match it and if not remove or resample them to ensure that the model is trained on data with the same sample rate.

If we have a model which requires fixed input shape, we also need to unify the length of all the data. If the audio record is longer we trim it, and if it is shorter pad it with values which will not turn it into noise. This step can be performed either on raw waveform or after the transformation to spectrogram.

3.2.2 Spectrograms

Sound and especially musical sound is a sum of signals oscillating at many different wavelengths. For instruments that have pitched sound, like flutes, pianos, violins etc., this is usually series of harmonics over the resonating frequency and the balance of these harmonics differentiates their timbres. For percussion instruments, their decomposition into individual audible frequencies is

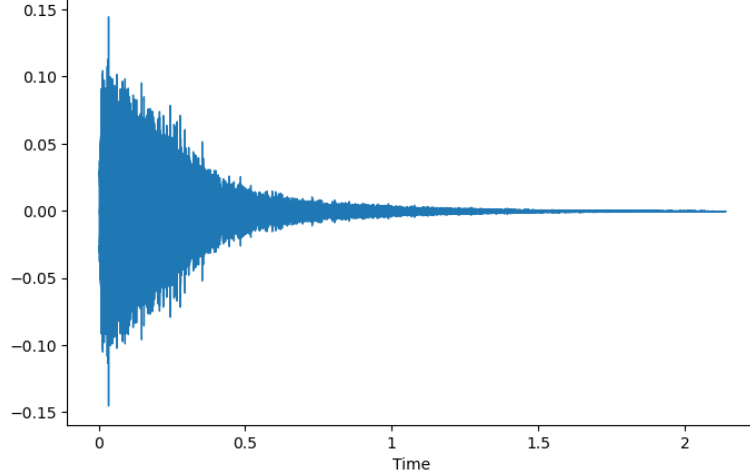


Figure 3.6 Audio wave of a crash one-shot

less predictable, because the resonance and dampening of percussion instruments is often only vaguely related to their pitch.

Let x be audio signal over time interval T , if we fix the T at arbitrary value t , we can analyse how the power or energy of $x(t)$ is distributed into frequencies F . This distribution is called the power spectrum or spectrum vector and we will denote it as $S_{x(t)}(f)$ for some frequency f . For continuous F , the power spectrum is a density function called power spectrum density. When we unfix the time, we will get function of frequency and time which returns power, $S_x(f, t)$.

However, we will consider discrete case, which means we will divide the frequencies and time into finite number of bins. We can represent it as matrix where columns will be indexed with time bins and rows with frequency bins. This matrix is called a *spectrum matrix*. A *spectrogram* then is a visualization of the spectrum matrix as a heat map with time on x-axis and frequencies on y-axis and the pixel intensity stands for power. We will use the terms “spectrogram” and “spectrum matrix” interchangeably, as is the norm in audio and music processing. We should note that if we fix time, that means take one column of the spectrogram, it does not has to be a distribution even though we defined it as one earlier, still the vector represents the amount of power over frequency, for example in decibel, or in short dB, which is a $\log(\text{power})$ and is often used in audio. Figure of spectrogram: 3.7.

There are various methods to obtain an spectrogram, but most of them require that the signal is first processed by the Short-Time Fourier Transform in short STFT 3.2.3. Because we want to train a VAE over spectrograms from which we then need to generate a waveform, we focus on methods that are invertible.

Nistal et al. [25] introduced an extensive overview and comparison of representations used in audio generation and synthesis, from both approaches: raw waveform and spectrograms. They trained a model based on GAN architecture for every method and compared them.

We also did our own research by trying out noise roundtrips of these representations, wave $\rightarrow$ spectrogram + noise $\rightarrow$ reconstructed wave. This experiments gave us understanding of how the representations behave when they are noised and understanding of what parameters do, and how to set them up to get in

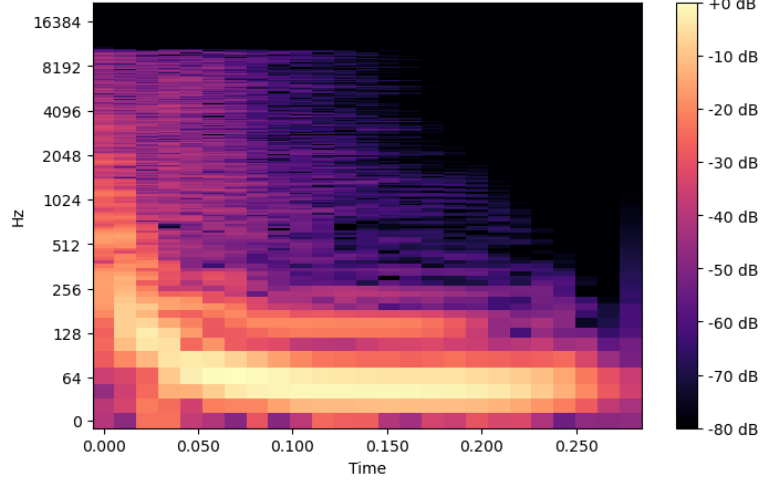


Figure 3.7 Spectrogram of a kick one-shot

order to get meaningful reconstruction with reasonable dimensionality. This was important because VAE has a tendency of noising all of the latent space, as a way of minimising the reconstruction loss.

Based on this research, the mentioned paper and other papers in 2, we decided to experiment with two representations STFT and MFCC.

3.2.3 STFT

Short-time Fourier Transform, abbreviated as *STFT*, introduced by Allen [26], is a processing method for non-stationary signals, such as audio signals, whose power spectrum is changing over time. The STFT method transforms the signal into a spectrogram, a time-frequency representation of the signal. STFT can be defined for a continuous case, but in practice the discrete version is used; therefore, we will explain the discrete version right away.

Let x be a discrete audio signal with sample rate sr . We can index $x(i)$ and it will return the i -th sample of x . STFT uses DTF - Discrete Fourier transform, that for a signal x will compute an average power spectrum over frequency bins. However, as we mentioned, the audio signal is non-stationary, so we would like to know how the power spectrum is changing over time. So we can split the signal x into smaller parts called frames and then compute the DFT for each of them, which would result in a spectrum matrix, or a spectrogram, where the i -th column is the power spectrum of the i -th frame.

$$S_x(t, k) = \sum_{n=0}^{N-1} x(n + tH) \cdot w(n) \cdot \exp(-i2\pi n \frac{k}{N})$$

where:

- x - audio signal.
- t - index of a frame and spectrogram column or a time bin.
- N - frame length, ideally equals to a power of two, so the Fast Fourier transformation algorithm can be used.

- H - hop length, number of samples between two consecutive frames. Often, it is defined as a fraction of the N .
- w - window function.

$\sum_{n=0}^{N-1} x(n) \cdot \exp(-i2\pi n \frac{k}{N})$ is the DFT algorithm, but we added a few features. Hop length H defines that frames can overlap, and the window function w usually makes the ends of a frame smoother. Both of these features ensure smoother transitions between the power spectrums of adjacent frames. Most used window function is Hann window and is defined as:

$$w(k) = \frac{1}{2} \cdot (1 - \cos(\frac{2\pi k}{K-1})), k \in \{1, 2, \dots, K\}$$

Where K is the number of samples in a frame. The plot of a Hann window and the result of applying it to a frame are shown in figure 3.8.

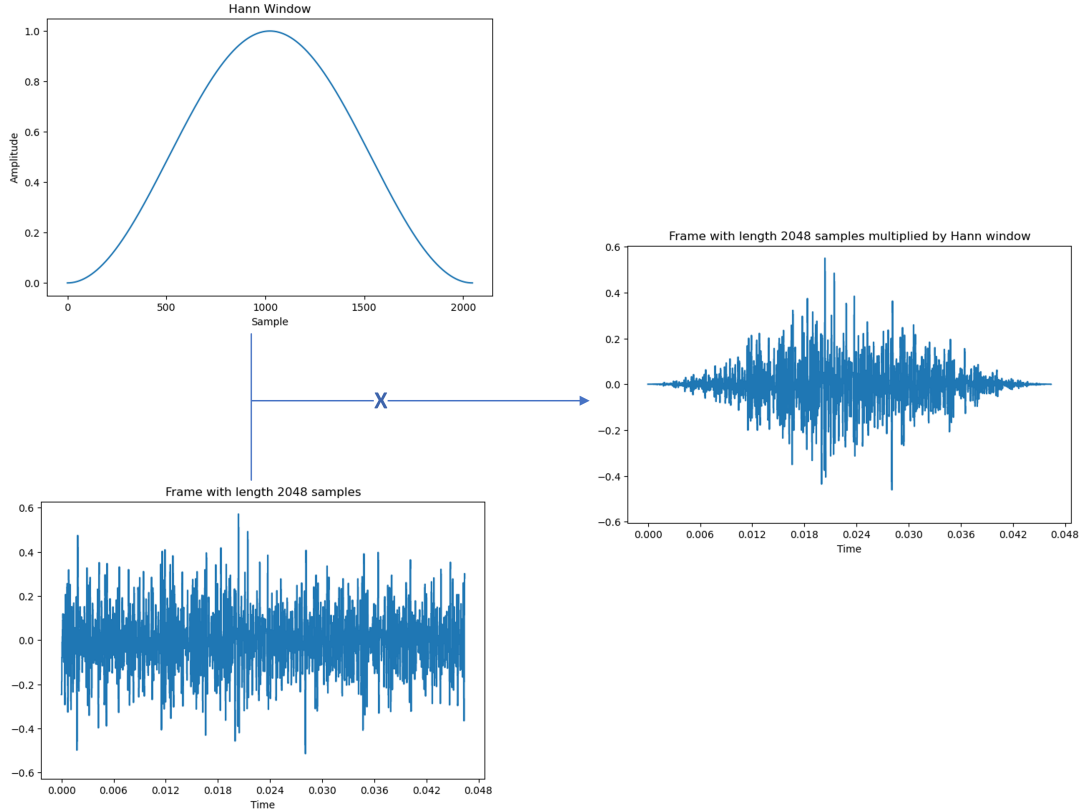


Figure 3.8 Hann window applied to a frame

The dimensionality or shape of input data is important for deep learning methods because some of them, including the ones we will be using, require a fixed input shape, and as the input shape increases, usually the training and inference time of the network rises too. Therefore, we will show the resulting shape of the STFT spectrogram based on the parameters of the STFT:

- Number of frequency bins or Height = $\lfloor N/2 \rfloor + 1$
- Number of frames or Width = $\frac{T-N}{H} + 1$, where T is the number of samples.

STFT is invertible by the *inverse STFT* algorithm, abbreviated as *iSTFT*, which is an important property for audio generation, as we mentioned earlier.

We mentioned that if the hop size is smaller than the frame size, the adjacent frames can overlap, and the bigger the overlap, the smoother the transitions between frames are. This parameter is important, especially when we want to transform the spectrogram back into an audio signal using iSTFT. The overlap can be computed as $overlap = 1 - H/N$, if $H \leq N$, and usually is shown in percents. Often, this value is chosen somewhere between 50% and 75%.

STFT returns a complex-valued spectrogram because it decomposes the signal into sinusoids that are represented by complex numbers. The stft spectrograms can be decompressed into two components:

- **Magnitude:** represents the amplitude, or the amount of power frequency f has at time t . In the context of complex numbers, it is the distance of a number from 0. Heat map visualisations of STFT spectrograms show the magnitude, like the one in 3.7. It can be computed as the absolute value of the whole STFT spectrogram: $Mag_x(f, t) = |S_x(f, t)| = \sqrt{Re(S_x(f, t))^2 + Im(S_x(f, t))^2}$.
- **Phase:** represents the offset of the frequency, and it indicates the starting point of the sinusoid at a specific time. In the context of complex numbers, it is the angle between the complex vector x axis in radians and can be computed as $Angle_{phase_x}(f, t) = \arctan(\frac{Im(S_x(f, t))}{Re(S_x(f, t))})$. The phase is crucial for accurate reconstruction of the signal out of the STFT spectrogram using iSTFT.

When we have these components apart and we want to join them back into a complex spectrogram, so we can apply iSTFT, we can transform the phase angle into a complex value on a unit circle and then multiply it by the magnitude:

$$S_x(f, t) = Mag_x(f, t) \cdot \exp(i \cdot Phase_x(f, t))$$

3.2.4 MFCC

Mel-scaled cepstrum coefficients, in short *MFCC*, introduced by [27], is a representation of an audio signal or, more accurately, a feature extraction method that is primarily used in speech processing but is used for musical audio as well.

The goal of MFCC is to represent the timbral features of an audio signal in a way that is similar to human auditory perception. An example of an MFCC spectrogram is shown in figure 3.9.

The pipeline that creates the MFCC uses multiple methods, as illustrated in figure 3.10. The first step is STFT, which we covered in the last section, but MFCC uses only the magnitude of STFT.

After that is the step “Log-Amplitude Spectrum”, also called *liftering*. When humans are speaking, the audio wave is created in two steps: first, there is an audio signal produced by the vocal chords called a glottal pulse, which is then modified by the vocal tract. We can set parts of the vocal tract, for example, the lips or tongue, into positions that then adjust the glottal pulse into syllables. When we are analysing human speech, the part of the audio signal that the vocal

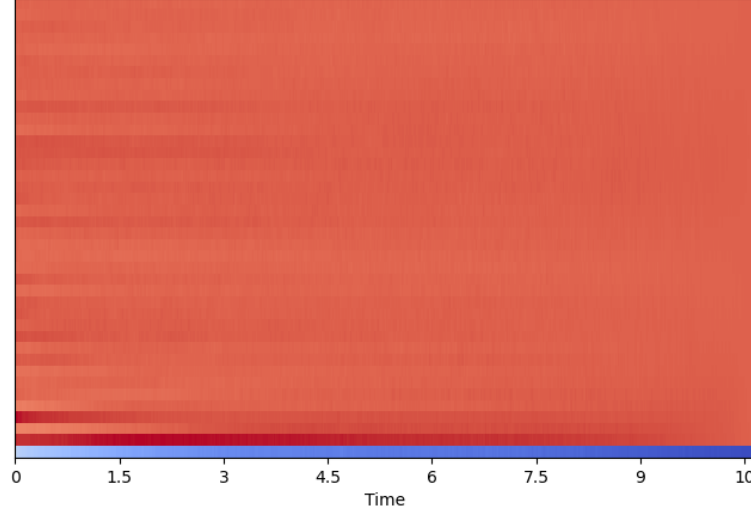


Figure 3.9 MFCC spectrogram

part modifies is what determines the syllables and words represented by the audio signal; therefore, we want to filter the glottal pulse out of the audio signal to remain with the part that the vocal tract makes. The “Log-Amplitude Spectrum” step is a mathematical function that approximately filters the glottal pulse. This step is optional and selectable with the “lifter” parameter. We will not be using it because we are analysing drum samples and not human speech.

The next step is mel-scaling. The mel-scale is a scale for how people perceive changes in frequencies. In lower frequencies, a change of 200 Hz results in a much bigger change in human perception of pitch than the same change in higher frequencies. In the last century, psychology experiments were held to determine the exact scale at which people perceive this change: $m(f) = 2595 \log_{10}(1 + \frac{f}{700})$; therefore, the pitch increases logarithmically with frequency.

The last step in the pipeline is a discrete cosine transform. In this step, we view the spectrum of the audio signal as another audio signal and analyse its spectrum. The result of this step is a spectrum of a spectrum that is also called cepstrum. The discrete cosine transform tries to approximate the spectrum of the audio signal with cosine functions, and it results in MFCCs. The first MFCC is the first cosine wave that approximates the spectrum the best, the second is the second best approximation, and so on.

The MFCC computation is only partially invertible because the phase computed by STFT is discarded and only the magnitude is used. The problem is with the phase of the STFT because, during the pipeline, we are throwing it away and only using the magnitude. In the next subsection, we will show how this problem can be bypassed.

3.2.5 Griffin-Lim algorithm

We mentioned in the STFT subsection that an inverse STFT transformation requires a phase. However, when we use MFCCs as an output representation of the generator, or more generally, in any situation where the phase is not available, we will have to compute it in a different manner. This is possible because phase

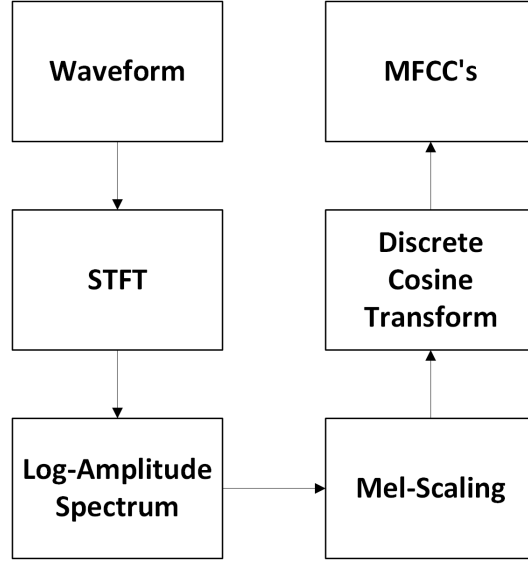


Figure 3.10 MFCC pipeline

and magnitude are not independent: the phase at a certain frequency will have implications for the magnitudes of that frequency along the sequence of time windows for all frequencies that do not resonate with the temporal resolution of the spectrogram. This is captured by a property of a spectrogram called *consistency*. In principle, a consistent spectrogram (a pair of one magnitude and one phase spectrogram) is one that can be obtained by using STFT on some audio signal. There are explicit consistency constraints introduced in Le Roux et al. [28], but the first definition will be sufficient for us.

Griffin and Lim [12] introduced an iterative algorithm for phase estimation or recovery that converges into a consistent spectrogram:

1. Input: magnitude Mag
2. Randomly generated phase $Phase_x^0$ and join it with Mag into spectrogram S_x^0 resulting in a signal x^0 .
3. Repeat until convergence, or until patience runs out.
 - (a) Apply STFT resulting in signal x^i
 - (b) Apply iSTFT and extract phase $Phase_x^{i+1}$
 - (c) Join Mag and $Phase_x^{i+1}$ together into S_x^{i+1}
4. Transform the last S_x^n into an audio signal x^{n+1} and return it.

The *Griffin-Lim algorithm* will not necessarily estimate a phase that results in the original signal, but in practice and also in our experiments, it has been quite successful. Figure 3.11 illustrates the resulting signal after 0, 5, 10, and 15 iterations.

As we mentioned in 3.2.3, the overlap parameter is important for the inverse transformation of the spectrogram to the wave. However, it is even more important when we recover phase with the Griffin-Lim algorithm before using iSTFT because

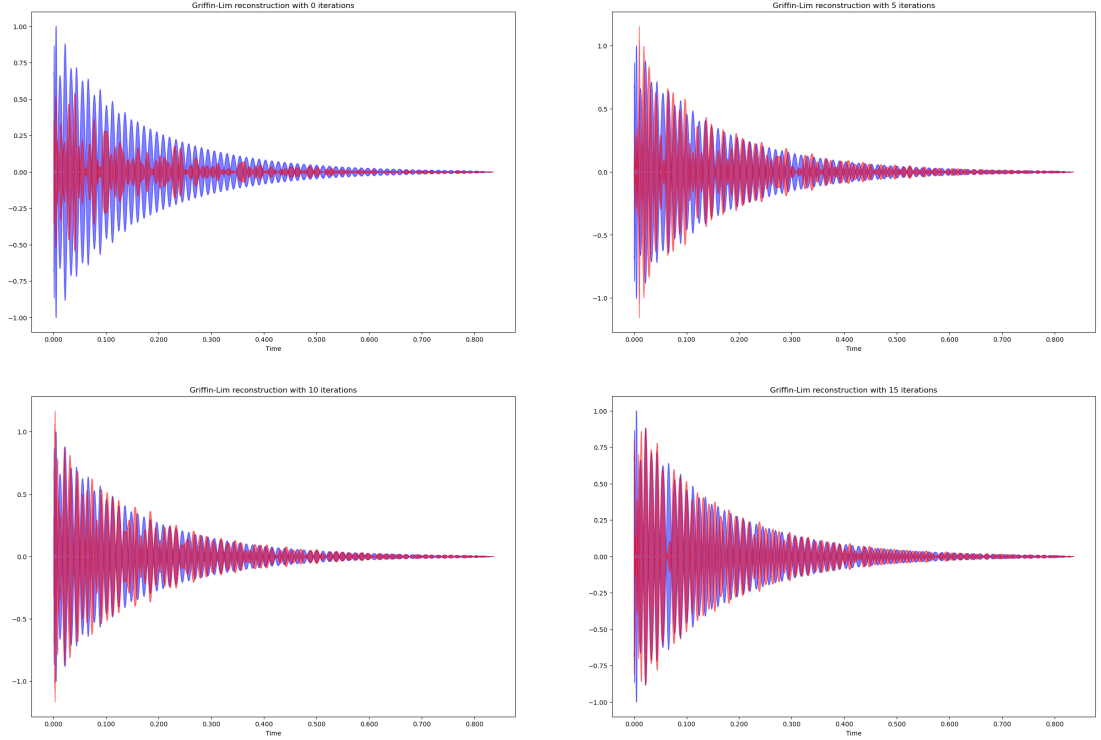


Figure 3.11 Griffin-Lim algorithm reconstruction after 0, 5, 10, and 15 iterations. Note that we used 32 iterations in our experiments, but we illustrated only iterations up to 15, because more iterations do not make a difference in the illustrations, but in the audio signal, they do.

a bigger overlap of adjacent frames makes transitions smoother, and thus the estimated phase will be closer to the actual one.

Why would we need to reconstruct the phase? We encountered two situations in which we had to use Griffin-Lim algorithm.

The last step of the inverse MFCC is the iSTFT. However, MFCC is computed out of STFT magnitude, and phase is thrown away; therefore, when generating new samples, we will only generate the MFCC representation without the phase, and during the inversion into the signal, we have to estimate it with Griffin-Lim.

When using STFT representation for training, the phase is hard to reconstruct and even harder to generate because even small changes in the phase can have a significant impact on the result. So we only generate the magnitude, and Griffin-Lim recovers the phase.

We used a library called Librosa, programmed by McFee et al. [29], where all of the representations and Griffin-Lim algorithm are implemented.

3.3 Encoder and Decoder Architecture

We described the VAE, but we did not identify what architecture we used for the encoder and decoder yet because the representation for input data was not clear in the VAE section. Now we have clarified that we will use spectrograms.

Spectrograms can be perceived as images with width and height represented by time and frequencies and with one channel for energy. Thinking about spectrograms as images has an important outcome. We can use architectures designed

for images, so the encoder will contain layers of 2D convolutional layers and the decoder will contain 2D transposed convolutional layers.

In the first models, we just used convolutional layers with a 3x3 kernel, stride 2 for reduction of width and height, and Relu activation. However, later, we got inspired by the ResNet architecture for image classification introduced by He et al. [30]. As in the paper, we used residual connections and batch norms for faster and more stable training and more effective backpropagation. We also tried the 4x4 kernel in the decoder's transposed convolutions to fight the chessboard pattern problem.

Using the well-known library Pytorch that Paszke et al. [31] introduced, we programmed all models and training functions in Python.

4 Dataset

This chapter is dedicated to the dataset, its organisation, and its birth. As probably mentioned earlier, the dataset was created by us. We will present the numbers, describe the structure, and explain the processes of gathering and data cleaning necessary for creating this dataset.

We attach the whole dataset used for training the models in the appendix.¹

4.1 Organization

We chose to organise the dataset in two directions:

1. drum types
2. genres

The first one, Drum types, divides the samples into nine categories describing the sample’s drum kit part (or other percussion) origin. This division can be quite challenging because of the effects and filters used on samples to make them sound more interesting, crazy, machine-like, etc. So any sample can sound very different from the sound it originates from. The samples may resemble a different category than the one they are assigned to, making categorization difficult. The drum-type categories are:

- kick,
- snare,
- tom,
- clap,
- crash,
- hat,
- ride,
- cymbal,
- other

We established the “other” category for less often used percussion instruments or samples whose category was not specified. In the same way, samples of cymbals whose precise cymbal type cannot be identified fall under the “cymbal” category.

The second division, genres, splits the one-shots into 11 categories that determine the electronic genre. This partition is even more uncertain than the first one because even the electronic track (that is what songs are called in electronic music) is tough to categorise due to the existence of tens or even hundreds of

¹All samples had to be converted from “.wav” to “.mp3” in order to fit them into the maximum size of the appendix. Original data are available on request.

electronic genres and frequent fusions of these genres. With this statement, it seems almost impossible to categorise a sample that is a tiny part of a track. That is why the sample counts for some of these genres are small. As a result of these obstacles, we did not use this division anywhere in this thesis. However, while creating this dataset, we thought these categories could be somehow useful, so the dataset contains them. The music genre categories are:

- DnB,
- Dubstep,
- EDM,
- Techno,
- Trap,
- House,
- Trance,
- Hip-Hop,
- Funk,
- Lofi,
- Unclassified.

Table of counts is shown in figure 4.1.

	clap	crash	cymbal	hat	ide	kick	snare	tom	other	Total
DnB	16	3	20	95	19	107	90	3	85	438
Dubstep	17	14	0	32	15	44	43	21	5	191
EDM	95	42	0	135	46	516	252	24	46	1156
Techno	32	0	20	121	0	119	92	14	157	555
Trap	71	33	8	100	35	154	117	14	134	666
House	151	23	30	250	26	277	231	23	123	1134
Trance	23	36	0	54	0	47	32	5	15	212
Hip-Hop	56	12	21	97	16	72	89	16	57	436
Funk	0	4	0	18	5	2	15	2	0	46
Lofi	19	2	0	41	0	26	30	0	62	180
Undef.	623	309	177	1757	225	2681	2069	790	2336	10967
Total	1103	478	276	2700	387	4045	3060	912	3020	15981

Figure 4.1 Table of counts for drum categories and genres categories.

4.2 Where samples come from?

Samples for electronic music can be obtained in several ways.

Some samples can be recorded from real acoustic or electronic instruments, vocals, or even random noises like the click of a computer mouse or the clinking

of wine glasses. Then these recorded sounds can be changed using various effects available in analogue machines, for example, synthesisers or drum machines. These machines can also make artificial sounds on their own. This way of music production stands at the beginning of electronic music and has created genres like disco, trance, techno, etc.

One-shots, loops, and melodies are often taken out of existing songs, and then they are used in new electronic tracks. This process is called sampling, was popularised by old-school hip-hop in the '90s, and is still used by many artists. However, this is considered a theft by many music artists and fans.

Nowadays, with the rise of computers and the internet, the most popular way of producing electronic music is using various programmes and huge sample libraries that can be downloaded or bought from the Internet. Numerous sites offer samples for sale or sometimes for free. This is also the way we chose to create my dataset.²

4.3 Data gathering

For several days, we surfed the internet and looked for free sample libraries (buying such amounts of samples would be quite expensive). Sometimes sites with products for electronic music production offer free sample packs for beginners as an advertisement for their site. Also, there are forums where people exchange their libraries or where starting artists provide their libraries to promote themselves. We found sites like these and downloaded almost 200 sample packs together, taking up to 70 gigabytes, approximately 100 hours of audio. For every pack, we recorded the site from which we downloaded it into a CSV file.

4.4 Data Cleaning

There is no defined structure for sample packs, every site and artist does it differently. Some packs are well organised with reasonable names of folders and files, but others are disordered. Manually going through 70 gigabytes of data, roughly 100 hours of audio, and listening to every sample seemed impractical. Therefore, we came up with several programmatic solutions to simplify the process. However, we still had to do some manual work because of the disordered structure.

Figure 4.2 shows steps used for data cleaning and organising.

Sample packs sometimes contain loops and melodies, as well as samples from other instruments, in addition to drum one-shot samples. In Step 1, we made a script that solved a portion of this issue. It involves three sections.

Section A chooses samples that are likely to be drum one-shots. Utilising regex, the algorithm matches patterns that indicate that the sample is a drum-one shot, and if the match succeeds, the sample is relocated to the “Uninspected Dataset”, a part where ready for inspection samples are stored. These patterns include titles of categories or the word “one-shot”.

Section B deletes samples that are improbable to be drum one-shots. This is not only done by regex pattern matching, as in the previous section, but also by

²Further information can be found here: https://en.wikipedia.org/wiki/Electronic_music

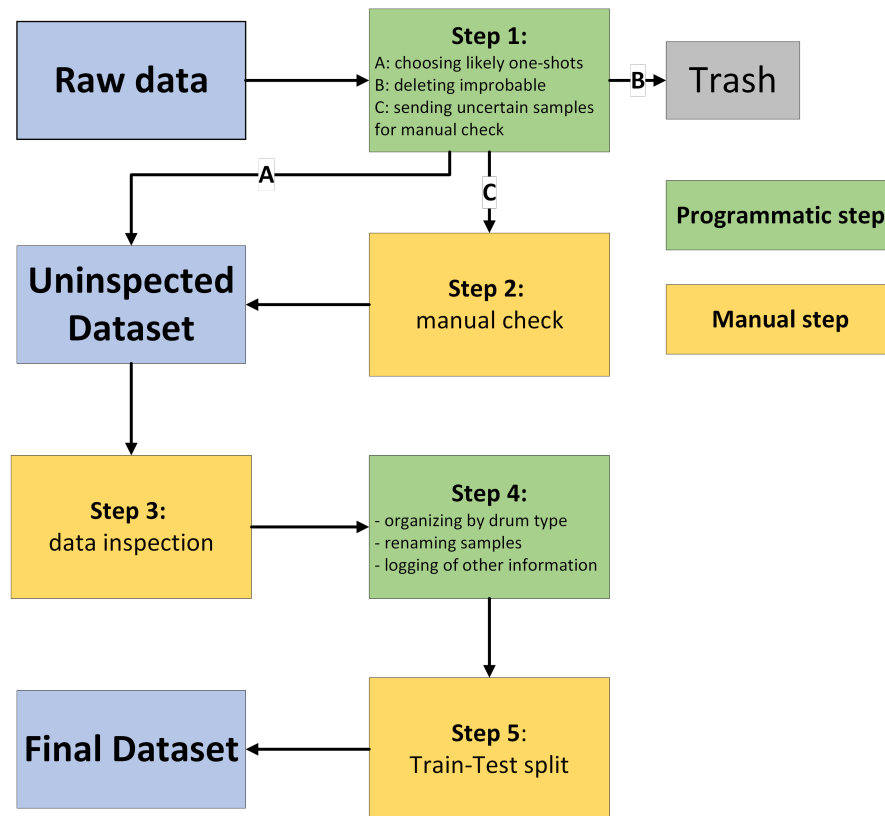


Figure 4.2 Diagram of steps used in data cleaning

inspecting file size in bytes and comparing it to a threshold. Patterns contain the words “loop”, “melody”, and names of other musical instruments. We chose a threshold of 1 000 000 bytes, or approximately 11 seconds of audio at a sample rate of 44.1 kHz, because it is improbable that a sample this length will be a drum one-shot.

Section C sends the remaining files to the next step for manual check.

After step 1, there remained less than 3 gigabytes of audio, which had to be manually cleaned in step 2. We did this by listening to the samples and deciding whether it was a drum-one shot. Sometimes the samples could be sorted by their names without listening to them. Even though its name was not matched by the previous step, it still carried sufficient information. After these steps, the “uninspected dataset” had almost 3.6 GB of drum one-shot samples.

The waveforms of drum one-shots can be naturally divided into two components:

- Hit: the start of the wave where power quickly rises and is caused by the hit of the stick or hand on the drum, cymbal, or other percussion.
- Decay: part of the wave where the power starts to decrease. Different types of drums have different behaviour of the decay, for example, the decay of a kick drum is short and the power decrease is steep, on the other hand, the decay of a crash cymbal will be opposite, long, and the power decrease is mild.

We can use this division to decide the drum type of a sample. For example,

the crash sample has hit with higher frequencies and longer decay in contrast to the kick sample, which has short decay and a low frequency hit.

In step 3, we listened to every sample to make sure that the first two steps selected true drum one-shot samples.

Step 4 is a script that organises the samples. First, it deduces its category and genre either by its name or it was already decided by us during the listening parts. Then it sorts them into directories based on the category and renames them into the format `{drum_type}_{number}.wav`. Lastly, the script created two CSV files: the first matched samples with the sample packs from which they came, and the second documented the genre for every sample, if it had one.

Finally, in step 5, we picked 10 or 20 representative samples from every category (the number depends on the size of the category) and relocated them into the test set. These samples were later used for the evaluation of our models.

5 Results examination

The evaluation of models is an important part of experimenting with machine learning. We somehow need to know which models perform better on the task and if the changes we apply have a positive impact.

However, the evaluation of generative models is not as straightforward as for different tasks. For example, in classification, there are multiple metrics describing the model's performance in various ways, and we can choose those that correlate the most with our goal. In recent years, even human evaluation might not define the results the best because many classification models overcame human performance.

In audio generation for electronic music production, we believe that an artist's auditory system, together with his personal taste, describes the model better than metrics used in this field. Therefore, our evaluation pipelines focus mainly on providing material for the artist's judgement to assess, and for that reason, we decided to name this chapter "Results Examination" instead of evaluation. In an application scenario, we would provide a selection of trained models with a series of samples (as described in the section "Human Examination" 5.2) and let the user select the model that best matches their preferences.

5.1 Automatic examination

Even though we put emphasis on human examination, it is time-consuming to examine every model, hence, we implemented some automatic methods that measure some properties of the model that indicate that the model will not be sufficient. This way, we can identify some unsuccessful models faster and invest our time and resources in those that seem more promising.

5.1.1 Loss functions

We can inspect the behaviour of the two components of the loss function. If we see that latent loss is at 0 and reconstruction loss is stuck at high numbers, we can deduce that the regularisation is not balanced, and the model will be generating noise averaged out of the training dataset.

On the other hand, if the situation is reversed and the latent loss is too high, the model will probably have problems generating new samples due to uncontinuous latent space.

However, sometimes these situations cannot be deduced from the loss values, and different approaches have to be used.

5.1.2 Distance Histogram

The automatic method we came up with is taking training data, feeding them to the encoder, and keeping means of the encoded distributions in the latent space. Then we can compute distance, we used Euclidean distance from the $\mathbf{0}$ vector and each one of the means and plotted them onto the histogram.

Thus, we can see how the distances are distributed, and similarly to the loss functions, we can see how well the regularisation is balanced. However, the losses are sometimes hard to interpret, but this approach shows us more clearly whether the means are close enough to the $\mathbf{0}$, so we can use random sampling from $\mathcal{N}(\mathbf{0}, \mathbf{I})$.

These histograms are illustrated in figure

5.1.3 PCA

Our dataset includes labels for the samples that represent the drum type of the sample. We are not using these categories anywhere in the model architecture, but samples from the same category should have similarities. Therefore, a good indication of a well-trained latent space is that the VAE encoder encodes samples in the same category into clusters.

These categories have non-empty intersections, for example, the “other” category contains samples that are similar to kicks, but during dataset creation we were sure enough to assign them to the kick category, so the latent space probably will not be perfectly clustered. However, it is still interesting to see how the VAE organises the latent space.

Visualisation of the latent space would be a great way to examine it, but visualising a space with many dimensions is difficult. Therefore, we will take the encoded means of all the training data, reduce their dimensionality to 3 dimensions, and then visualise them on a 3D plot. We will do this reduction with a method called PCA.

Principal component analysis, in short *PCA*, introduced by Wold et al. [32] is a dimension reduction method that projects the input data in n -dimensional space onto a subspace with k -dimensions, k can be chosen arbitrary but $k \leq n$. The PCA finds a projection that preserves the maximum variance in the data, or equivalently, the projected data has minimal MSE with the original data.

The method does this by constructing the covariance matrix C of the input data, $C_{ij} = \text{Cov}(x_i, x_j)$ and this matrix is symmetrical. It can be proven that the k dimensions preserving the maximum variance are generated by eigenvectors $v_1, v_2, \dots, v_k$ of the C matrix that belong to the k biggest eigenvalues $\lambda_1, \lambda_2, \dots, \lambda_k$.

There are two ways to find the eigenvalues and eigenvectors of a symmetrical matrix.

Every symmetrical matrix has spectral decomposition $C = Q\Lambda Q^T$ where Q is an orthogonal matrix whose columns are eigenvectors of the matrix C and Λ is a diagonal matrix with eigenvalues on the diagonal. By finding this decomposition, we will find the eigenvalues and their eigenvectors, so we will just take the k eigenvectors corresponding to k biggest eigenvalues and put them as rows in a new matrix P , which is a matrix of projection into our desired subspace. Now we take all of the training data and, by matrix multiplication with P project them into that space. However, finding the spectral decomposition of a symmetrical matrix is $\mathcal{O}(n^3)$ which can be long for a massive dataset.

The second option is using an iterative algorithm for finding eigenvalues that, for small k works much faster than spectral decomposition.

Finally, we can take projected data and plot them on the graph, where we will assign different colours to every category and observe how the data are distributed in the latent space. When we see that the points are sparse, or all the points were

projected onto one point, or the data are not clustered at all, we can assume that there is something wrong with the model.

5.1.4 Fréchet Audio distance

Lastly, there is a metric called *Fréchet Audio Distance* that measures similarities between the distribution of real audio features and generated ones, introduced in Kilgour et al. [33]. This concept is based on the Fréchet Inception Distance used for evaluating generated images. Fréchet Audio Distance is widely used in audio generation and synthesis tasks.

Fréchet Audio Distance works on a deep learning model pretrained on a vast dataset of audio samples, usually based on a convolutional architecture. The model learns to extract high-level features that characterise the audio sample.

Real and generated data are fed into the network, resulting in the extracted features, from which we compute a mean vector and covariance matrix for both real and generated data. Finally, we compute the Fréchet Distance between normal distributions with the parameters we computed before and return it.

However, we did not use this metric because we did not manage to find a suitable model for extracting audio features, and creating one ourselves was too complex with the resources we had. And anyway, if we only train some model for computing Fréchet Audio Distance on our dataset, we are not introducing information beyond the scope of the actual task that we want to evaluate.

5.2 Human examination

In the next subsections, we will introduce several methods for obtaining samples for human examination. But before that, we establish the properties of the sound that a human tester should be looking for:

1. Noisiness: the VAE has a tendency to create noise over all of the samples in order to minimise the reconstruction loss, mainly when MSE is used. Therefore, human testers should definitely focus on this property.
2. Weird sound artefacts: Sometimes, especially sounds with higher frequencies, tend to have weird artefacts after reconstruction.
3. Likeness to the desired category: a successfully trained VAE should be able to sample sounds from a selected category or even sample a combination of these categories that resemble the category.
4. personal opinion: a significant part of music is the personal preference of the listener or artist; therefore, this should be included in the testing parameters.

5.2.1 Reconstruction

An almost perfect reconstruction of train and test data is a necessity when we are training a model. If our reconstruction is noisy and has artefacts, then the probability that the model will generate quality samples is very low.

Therefore, the first step in our evaluation pipeline is listening to reconstructed samples from the train and the test set.

From the train set, there are randomly chosen train samples that are reconstructed and prepared for listening. This shows the overall reconstruction capabilities of the model.

Samples from the test set are representants of the categories that the model has not seen; therefore, they are testing generalisations of the model. Moreover, these samples are the same for all of the models, so they are used for testing reconstruction between models.

5.2.2 Sampling from the latent space

The second method is the usual usage of the VAE. The latent loss enforces the encoded distributions to be closer to $\mathcal{N}(\mathbf{0}, \mathbf{I})$ and with well balanced regularisation, this results in the majority of the distributions' means being near $\mathbf{0}$, and variance around $\mathbf{1}$, so when we are randomly sampling from the prior $\mathcal{N}(\mathbf{0}, \mathbf{I})$ there is a high probability that we will generate something meaningful.

We are generating 5 samples from the prior, and then we are increasing the variance to see where the generating still makes sense and where the noise barrier starts.

5.2.3 PCA shift

We described the PCA method in 5.1.3 for latent space visualisation, but we find out that there is another utilisation of this method. As we explained, the subspace that PCA computes preserves the most variance in the data; therefore, the directions of these dimensions represent some big changes in the data. For example, the first dimension can represent the length of the sample, the second one the power, and the third one the pitch. It is not guaranteed that these dimensions will have such a meaning, but empirically, it happens often.

We can extract the eigenvectors defining the main dimensions and normalise their norm to one. Then we take a point in latent space $\mathbf{z}$ and try to shift it by the eigenvector $\mathbf{v}$ multiplied by some alpha $\mathbf{z}' = \mathbf{z} + \alpha \cdot \mathbf{v}$, and then decode it with a decoder.

We chose the first principal components (eigenvectors) and alpha values ranging from -5 to 5, and then shifted multiple times a representant from every sample group. This method tests whether principal components have some high-level meaning, whether the latent space is continuous, and where the noise barrier starts.

This method is very important not only for evaluation but also for future use in music production because, in a well-trained latent space with meaningful principal components, it gives the artist an interactive way to adjust the sample in a way he wants; therefore, finding the desired sample for his track.

5.2.4 Convex combinations

The last method for generating samples we implemented into our evaluation pipeline was convex combinations. A *convex combination* is an expression:

$$z = \sum_{i=1}^n a_i \cdot v_i$$

where $v_1, v_2, \dots, v_n$ are vectors, $a_1, a_2, \dots, a_n$ are coefficients that must fulfil these conditions:

$$\forall i : a_i \geq 0$$

$$\sum_{i=1}^n a_i = 1$$

Vector $\mathbf{a} = (a_1, a_2, \dots, a_n)$ is basically a probability distribution over the vectors v_i .

The geometric view is that the vectors v_i are vertices of a multidimensional polyhedron, and z is any vector inside of it for some combination of a_i that fulfils the conditions.

A well-trained VAE should cover the whole latent space with normal distributions, but that indicates that these normal distributions will have overlaps. Multiple training data could be sampled from these overlaps during the training, so the VAE has to average them in order to satisfy the reconstruction loss. Therefore, when we sample from these overlaps, we could get new data that is a combination of two or more training data as output. This is illustrated in the middle picture in figure 3.5.

Convex combinations are a great way to find these overlaps in latent space and sample from them. We choose two or more training data, encode them into the latent space, and out of their means, compute a convex combination z which is a point somewhere between them. Then we decode this z with the decoder, which results in a new datum that should contain certain elements from all of the data that participated. We can choose coefficients a_i arbitrary as long as they fulfil the conditions, but those vectors whose coefficients are bigger will participate more in the result.

During the testing of the model, the evaluation pipeline generates a few convex combinations between pairs of data chosen out of the five biggest one-shot categories. Then we listen to them and decide how well these combinations resemble the categories.

6 Architectures, Experiments and Results

In this chapter, we are going to present our experiments and their results. Usually those two chapters are apart, but we have decided to fuse them together because our evaluation is based mainly on human examination; therefore, our results are a review made by the developer as the experiments finish, and they are used as guidance for gradually improving the models.

Our goal is to create an interactive tool for electronic music production; therefore, examination by listening to the generated samples from the model is crucial. It is necessary to examine unconditionally generated data, not only cherry-picked samples we liked, in order to understand what the model does and determine if it is successful.

Therefore, for the most successful experiments, we attached the results of the examination pipeline in the appendix, which means visualisation plots, training logs, the configuration file, and most importantly, the generated data we listened to during the inspection of the models' performance. In the text, after an experiment, there will be a footnote specifying the directory¹.

Additionally, we attach the whole dataset used for training the models²

We will start by introducing the specific VAE architectures used for experimenting, continue to the successful experiments, divided into two subsections based on the representations we use, MFCC and STFT, and finally we will also report experiments that failed.

6.1 Architectures

As we mentioned earlier, our model of choice is the VAE. For the encoder, we use 2D convolution layers, and for the decoder, we use corresponding transposed convolution layers that we call deconvolution layers. We will mainly describe the successful models that advanced our research the most.³

Experiments started with a small model called VAE_1, with only two layers with a 3x3 kernel and stride 2. We used this model for debugging the training script, the loss function, and other functions necessary for training a deep learning model and for testing how the VAE architecture would behave on a small portion of our data. We did not use this model for future experiments because the model's capacity was not large enough to do anything useful.

Next, we designed a much bigger model, VAE_2 based on similar layers as the small one, but we got stuck on several problems. We will describe the model and its problems in more detail in section "Failed Experiments" 6.3.

¹The text form of the thesis does not enable the key qualitative inspection of the data in the body (the required PDF-A format does not support the insertion of audio recordings); therefore, we selected the least bad solution

²All samples had to be converted from ".wav" to ".mp3" in order to fit them into the maximum size of the appendix. Original data are available on request.

³Names of those models will align with real names in the git repository of this thesis.

6.1.1 VAE_3

The first successful model was a smaller version of the second model with fewer layers and channels. This model was created for the MFCC representation with input size $(height, width, channels) = (256, 112, 1)$. The encoder and the decoder almost mirror each other's layers and input/output channel counts, up to a few small differences described below.

The encoder consists of 4 convolution blocks, a fully connected block, and 2 fully connected layers for mean and logvar. that serve as the latent representation of the VAE (see figure 3.4) .

The initial block consists of 1 convolution layer and ReLU activation. Convolution layers have a 5x5 kernel, 1 input channel, 8 output channels, stride 2, and padding 2.

The second block consists of two convolution layers and ReLU activations. Both convolution layers are the same: 3x3 kernel, 8 input and output channels, stride 1 and padding 1.

The last 2 convolution blocks are the same, containing 2 convolution layers and Relu activations. The first convolution layer has a 3x3 kernel, stride 2, padding 1, and doubles the channels. The second convolution layer is the same as the convolution layers used in the second block.

Moving to a fully connected block that has a flattening layer, one fully connected layer that reduces the dimension to the latent space dimension, and Relu activation.

Two last layers are fully connected layers that perform linear projection without changing the dimensionality, resulting in two vectors, the mean and the logvar, of an input's distribution in latent space.

As we mentioned, the decoder almost mirrors the encoder, except for a few characteristics. The two fully connected layers for mean and logvar because the input of the decoder is a sampled vector from the latent space. Deconvolution layers with stride 2 need output padding 1, so the dimensionality stays the same. And lastly, convolution blocks with the stride and convolution blocks without the stride are exchanged.

A diagram of this architecture is shown in figure 6.1.

6.1.2 VAE_4

VAE_3 had decent results, but the problem was the low capacity of the model, because even with multiple experiments without latent loss, we could not decrease the reconstruction loss enough. Therefore, we created a bigger version, VAE_4 which is similar but has more convolutional layers and blocks.

The convolution layer in the initial block has a 7x7 kernel and padding 3. To every block except the initial one, we added a copy of the second convolution layer and added a whole new convolution block, which is a copy of the two last convolution layers. Input and output channels stay the same.

VAE_4 has several submodels that implement various modifications and features we wanted to test, but the core stays the same. We will discuss these modified versions in future sections where individual experiments will be discussed. A diagram of this model is shown in figure 6.2

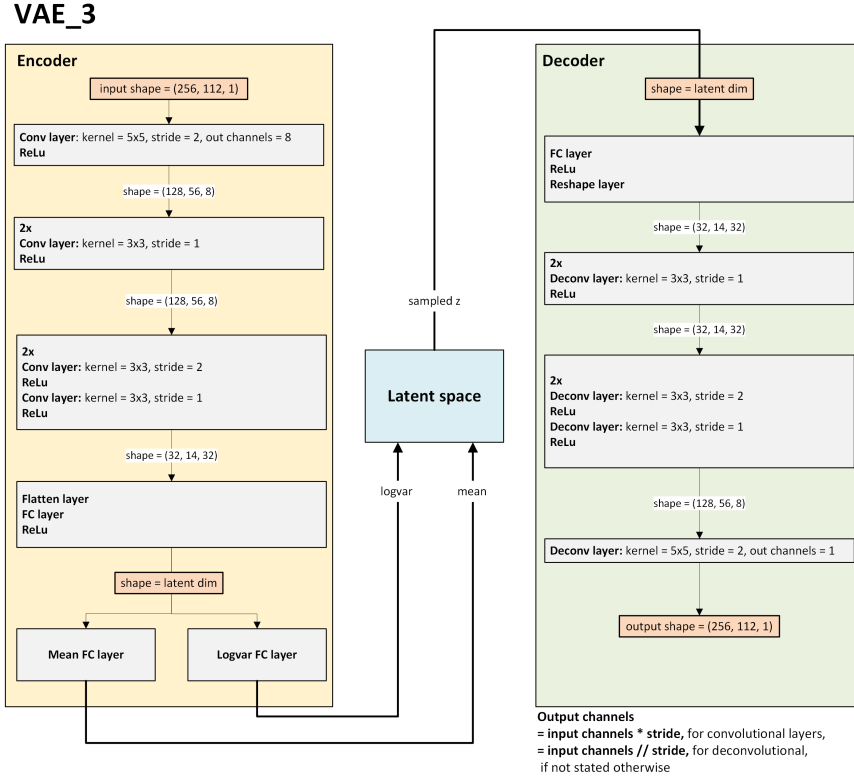


Figure 6.1 VAE_3 network architecture

6.1.3 VAE_magpie_1

VAE_4 had better results than his predecessors, but it still did not have the reconstruction quality we wanted due to insufficient capacity and slow training. Therefore⁴, we drew inspiration from the enormous image networks, especially residual connections in the ResNet architecture introduced by He et al. [30].

Residual connections, also known as skip connections, allow the input to bypass one or more layers and then add it to the output of these layers. This is an important concept that helps to solve the problem of vanishing or exploding gradients; therefore, allows us to create deeper networks while retaining fast and stable training. We will mention vanishing and exploding gradients later, in 6.3.

The architecture we designed is called VAE_magpie_1, and it performs the best out of our architectures. From the beginning, the model was designed for the STFT representation in contrast with its predecessors, so the input shape has changed to $(height, width, channels) = (257, 256, 1)$.

Both the encoder and the decoder consist of several residual blocks; the blocks are the same in principal, but in the decoder's residual blocks, deconvolution layers are used instead of the convolution ones. The block takes several arguments, *kernel_1*, *kernel_2*, *stride* and input/output padding. The padding is designed to preserve the shape of the input if no stride is used or $\lceil inputshape/2 \rceil$ if stride 2 is used; this only applies to width and height, not channels. We will not discuss the specific values of the padding⁵.

⁴With thanks to RNDr. Milan Straka, Ph.D. for consulting the model issues with us

⁵The exact settings can be found in the git repository of this thesis.

VAE_4

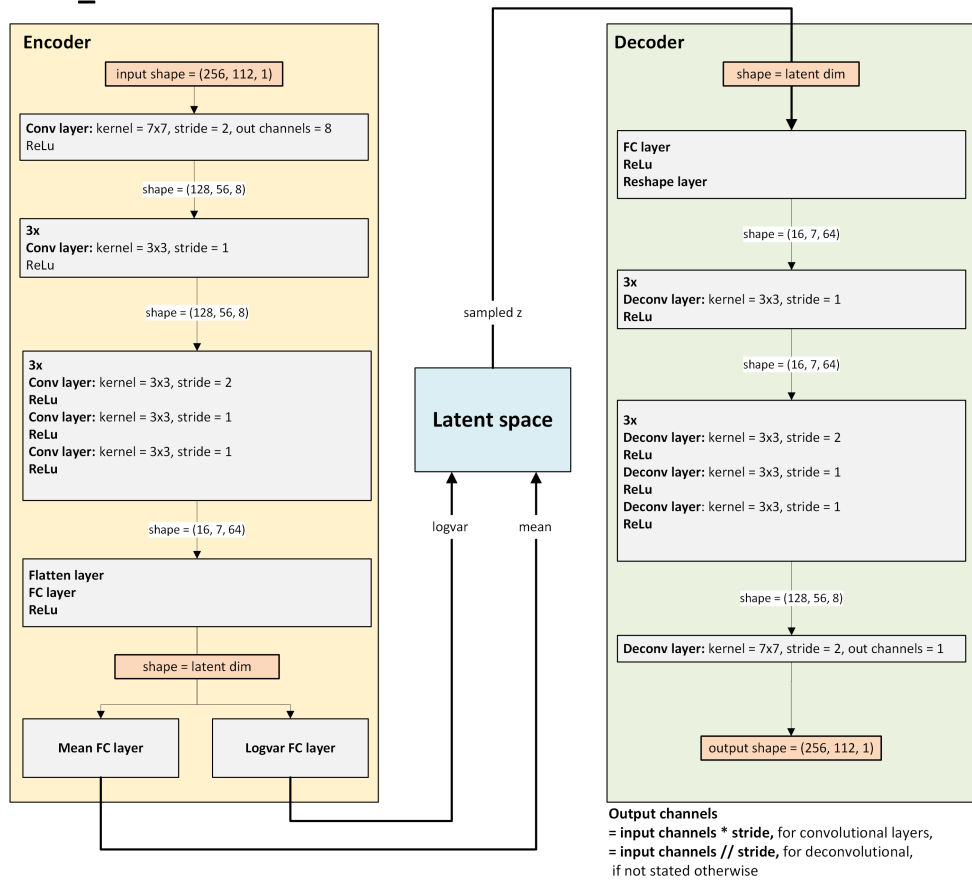


Figure 6.2 VAE_4 network architecture

The residual block input splits into two paths; the left path is residual and it consists of 1 convolution layer (or deconvolution) with 1×1 kernel, $\text{stride} = \text{stride}$ and $\text{outputchannels} = \text{inputchannels} * \text{stride}$ (or $\text{outputchannels} = \text{inputchannels} * \text{stride}$ for deconvolution layers) if not defined otherwise, followed by batch normalisation layer, in short batch norm. This path is what makes this block residual. The convolution layer with a 1×1 kernel and stride gives the path the ability to change shape, so the outputs of these paths can be added together.

The right path contains two components. The first component consists of a convolution layer with $\text{kernel} = \text{kernel}_1$, $\text{stride} = \text{stride}$ and $\text{outputchannels} = \text{inputchannels} * \text{stride}$ (or $\text{outputchannels} = \text{inputchannels} * \text{stride}$ for deconvolution layers) if not defined otherwise, followed by a batch norm and ReLU activation. The second contains a convolution layer with $\text{kernel} = \text{kernel}_2$, $\text{stride} = 1$, and keeps the number of input channels, followed by a batchnorm.

At the end, these two paths are added together and fed into a ReLU activation. Illustrations of these blocks are in figure 6.3.

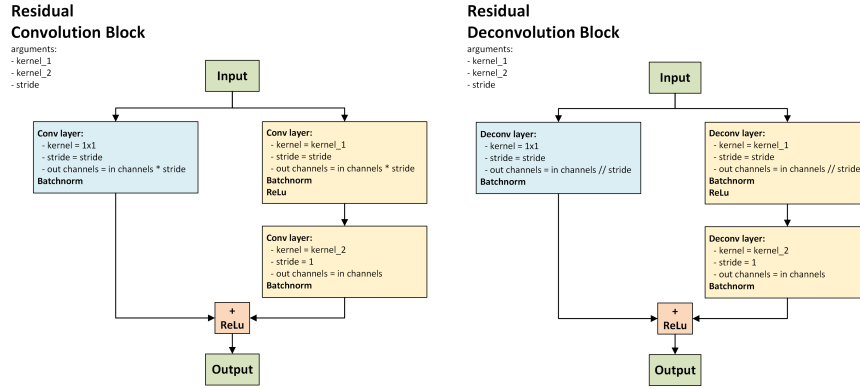


Figure 6.3 Residual convolution and deconvolution blocks used in VAE_magpie_1.

The encoder starts with an initial residual convolution block with $\text{kernel}_1 = 3 \times 3$, $\text{kernel}_2 = 3 \times 3$, $\text{stride} = 1$ and $\text{outputchannels} = 16$, followed by four same blocks, which consist of two convolution residual blocks where the first has $\text{kernel}_1 = 3 \times 3$, $\text{kernel}_2 = 3 \times 3$, $\text{stride} = 2$ and the second $\text{kernel}_1 = 3 \times 3$, $\text{kernel}_2 = 3 \times 3$, $\text{stride} = 1$. Afterwards, it is the same as in previous models: a flattening layer, an FC layer that reduces dimensions to latent space dimensions, ReLU activation, and two FC layers, one for the mean and one for the logvar.

The decoder starts with the FC layer, which rises the dimensionality to $17 \times 16 \times 256$, the ReLU activation layer, and the Reshape layer, which reshapes the input to $(17, 16, 256)$. Next, there are four blocks that consist of two residual deconvolution layers, where the first has $\text{kernel}_1 = 4 \times 4$, $\text{kernel}_2 = 4 \times 4$, $\text{stride} = 1$ and the second has $\text{kernel}_1 = 4 \times 4$, $\text{kernel}_2 = 3 \times 3$, $\text{stride} = 2$. Then one residual deconvolution block with $\text{kernel}_1 = 4 \times 4$, $\text{kernel}_2 = 4 \times 4$, $\text{stride} = 1$ and lastly the final deconvolution layer with $\text{kernel} = 3 \times 3$, $\text{stride} = 1$, and $\text{output channels} = 1$.

The encoder and the decoder mirror each other, except for the 4×4 kernels in residual deconvolution blocks and the final convolution layer at the end of the decoder. 4×4 kernels for balancing the chessboard artefact, which is a common problem with deconvolution layers.

Each input pixel of a deconvolution layer affects several output pixels based on a kernel shape. And if we use a stride bigger than one with odd kernel sizes, then some output pixels will have a different number of input pixels affecting them, which results in a chessboard-like pattern at the output. This issue can be solved by using an even kernel size in layers with a stride bigger than one, because then the output pixels have a more evenly distributed number of input pixels affecting them.

The final convolution layer is there because we do not want the network to finish the computation with ReLU activation at the end because some spectrogram representations contain negative values, and with ReLU activation at the end, we could not reconstruct them.

This network architecture is illustrated in figure 6.4.

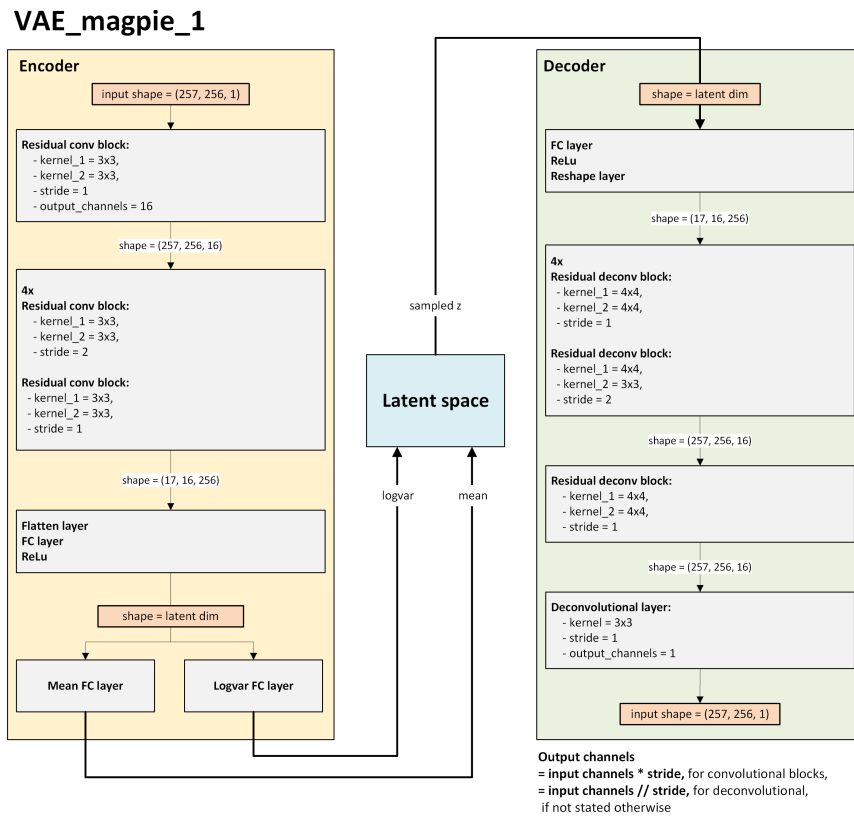


Figure 6.4 VAE_magpie_1 network architecture

All convolution layers are initialised with the “He initialization” introduced by He et al. [34], which is a common initialization method used for convolution layers with ReLU activation. The authors of this method are the same who introduced ResNet, which uses this method too.

Fully connected layers are initialised with default pytorch initialisation, which is a uniform distribution $U(-std, std)$ where $std = 1/\sqrt{weight_size}$, except the logvar fully connected layer.

In the first epoch, after 2 or 3 updates, the logvar exploded into values 100 and more, which is a problem because during sampling from the encoded distribution, we have to use the exponential function on the logvar in order to get the variance, but with values that big, the exponential made infinity or minus infinity out of it.

Afterwards, the whole training started diverging. This problem occurred especially in models where we were testing reconstruction capability; therefore, we turned off the latent loss, which is dragging the logvar down during updates. This problem occurred in models with latent loss as well, but this was less frequent.

We solved this problem by initialising the weights of the logvar FC layer with values from the normal distribution $\mathcal{N}(0, 0.00001)$. This way, the logvar did not explode, and the training became stable.

6.2 Experiments

In this section, we are going to explain the successful experiments. They are divided into two sections based on the spectrogram representation that was used in them; however, the order of this division will approximately correspond to the timeline in which we conducted these experiments because we started with the MFCC representation, then tried the STFT and found out that results with the STFT were better, so we kept this representation for the rest of the experiments.

We started with the MFCC representation because it is the default representation used in the majority of the audio processing tasks. Additionally, it had good results in the overview paper [25] about audio representations used in audio generation tasks, from which we drew inspiration.

Every experiment consists of several trained models, usually around 10, because with every new feature we were trying, it usually changed the circumstances; therefore, we had to find the right value of the KL divergence regularisation⁶ again. Additionally, we were trying models with and without the addition of noise.

Results for successful and interesting models will be attached to supplementary material of this thesis, and after each experiment there will be a footnote specifying the directory. From the results for each experiment, we only selected a few models with the best results to save time and space.

Computational resources were provided by the e-INFRA CZ project (ID:90254), supported by the Ministry of Education, Youth, and Sports of the Czech Republic.

6.2.1 MFCC

First, we conducted a series of roundtrip experiments: wave $\rightarrow$ spectrogram (+ noise) $\rightarrow$ wave, to set the right parameters for the MFCC method and noise function. Our goal was to find a reasonable input shape for the model while preserving an almost unperceptible difference between the original wave and the reconstructed one. Secondly, we wanted to decide the type of noise that we would use (multiplicative or additive) and what parameters it should have (distribution, variance, and mean).

The noise functions we programmed were divided into two categories:

- additive noise: a method where the noise is added to the spectrogram
- multiplicative: a method where we sample a coefficient for every pixel, and then multiplied the spectrogram with these coefficients.

⁶Note that the KL regularisation is actually inverted KL regularisation; therefore, when we want to regularise the KL divergence more, we have to decrease the regularisation.

Then we tested two distributions: normal and uniform, from which we chose the normal distribution.

The MFCC parameters we varied were the number of MFCCs, the number of FFT frequencies, and hop size. Other parameters stayed default, as defined in the python Librosa library, because changing them usually worsened the reconstruction. The number of MFCCs and number of fft affect the height of the resulting spectrogram; we set them to $n_{mfcc} = 512$, $n_{fft} = 512$, resulting in a height = 256. The hop length affects the width of the spectrogram, and it was set to 256. The resulting spectrograms had to be trimmed or padded with values along the time dimension because the width of the spectrogram depends on the length of the sample from which it was computed. We decided to use 112 as the width of the spectrogram, which is around a second of audio with a hop length = 256 and a sample rate = 44.1 kHz. We also had to test with what values we would pad the spetrogram. We tried all zeros, but that generated loud noise, so we tested padding it with values from the last column of the spectrogram, and this worked well.

6.2.2 VAE_3 experiments

So when we started experimenting with the bigger VAE_3 architecture and with the whole dataset, we had 4 goals. First, finding the quality limits of the reconstruction with this model; second, testing how different values of KL divergence influence the results and how to set them up in order to get a balanced latent space; third, testing how the model and results will behave when we train the model on noised inputs; and last, set a comparison bar for future experiments.

With these goals in mind, we conducted experiments with 5000 epochs, a 0.0001 learning rate, and two levels of noise: none and multiplicative normal noise with $mean = 1$ and $std = 0.05$.

The first experiment⁷ was targeted at the first goal: reconstruction, so we set the KL regularisation to 0, thus turning off the latent loss entirely.

This model was able to reconstruct the volume and decay of the sample, which is a good starting point. Surprisingly, there was not much difference between the noised and unnoised models; both produced similar reconstructions. However, we encountered multiple problems: the model produced noisy reconstructions, and in particular, it created a strange “noise artefact”, especially in samples with higher frequencies, for example, cymbals, and less common sounds were not reconstructed at all; instead, the model returned average noise. (All of the sampling methods failed, which was expected due to the latent loss being turned off.)

In the second experiment,⁸ we set the KL regularisation to 0.1 and 0.05⁹. The model with 0.1 KL regularisation was bad; the reconstructions almost did not remind the original samples. The second value did better, but the reconstruction was not as good as in the previous experiment without the latent loss, (which was expected). The main reconstruction issue was that many similar samples

⁷The results of this experiment are in folder “results/VAE_3_kl-reg=0_low-learning-rate_epochs=5000” of the supplementary material.

⁸The directory with the results: “results/VAE_3_low-learning-rate_epochs=5000”

⁹We did not know how many experiments we can run in parallel on the “Metacentrum”, a project mentioned at the end of the “Experiments” section’s introduction part 6.2; therefore, we only tried these two values, later we were trying 4 or more values at the same time.

got decoded into the same distribution in latent space; therefore, a spectrogram decoded from this distribution is an average of these samples. Sampling from prior was also problematic because it rarely generated anything other than noise. However, the other sampling methods did work better. The first PCA component was increasing and decreasing the volume, and the second was controlling the length of the decay, even though these changes were very subtle. Convex combinations generated interesting combinations for mergers of kicks, toms, and snares but did not work for higher-frequency samples, cymbals, and claps. Another good result was that samples from the same drum category were usually encoded close to each other in the latent space, thus forming nice clusters. and the peak of the distance histogram was around 1.

All taken into account, these were highly encouraging results because they demonstrated to us that the main principles of VAE architecture could work for our data and expanded our knowledge about the problems we will be facing.

After these experiments, we determined 3 new directions we want to continue in: new features often used in deep learning, lower KL regularisation, and a bigger model.

In the first direction, we conducted several experiments on the VAE_3 architecture that ended up with failure or worse results than these experiments. These will be mentioned in the section 6.3.

In the second direction, we tried a new batch of experiments with a bigger batch size 64, 128, a lower and wider range of KL regularisation values: 0.2, 0.1, 0.05, 0.01, 2000 epochs, and a learning rate 0.001. These experiments resulted in slightly better and less noisy reconstructions, although with the same noise artefact. Performance in sampling methods got better as well: PCA had a stronger effect, convex combinations were less noisy, and sampling from the prior generated some more samples that were not just noise. Therefore, we found out that the strength of KL regularisation will be the most impactful training parameter.¹⁰

After a series of experiments on VAE_3 we decided that the capacity of that model was insufficient and we needed to build a bigger one. Therefore, we designed VAE_4 and performed the same series of experiments on it, except we tried another experiment with an even bigger batch size: 128, 256 lowered the KL regularisation: 0.05, 0.01, 0.005, 0.001, increased the number of epochs to 10,000, and decreased the learning rate to 0.0001. This series resulted in a little better performance, but we still encountered the same problems: noise in the background, noise artefact in cymbals, and similar samples being encoded into the same distribution.¹¹

Therefore, we decided to try different spectrogram representation.

6.2.3 STFT

We started with the same process as with MFCC, testing parameters of the STFT and noise behaviour on roundtrips. In STFT, the parameters that change

¹⁰The directory with the results:

“results/VAE_3_epoch=2000_bigger-batch”

¹¹The results of these experiments are in folders: “results/VAE_4_kl-reg=0_low-learning-rate_epochs=5000”, “results/VAE_4_low-learning-rate_epochs=5000”, “results/-VAE_4_epoch=2000_bigger-batch”, and “results/VAE_4_epoch=10000_bigger-batch” of the supplementary material.

the resulting shape are frame size and hop length, so we varied these. Other parameters can change the reconstruction quality, but we stayed with the default ones based on our research on roundtrips and papers that used this representation (see 3.2.3).

As we mentioned in 3.2.3 frame size N and hop length H together form an important parameter called overlap, which is usually set in range from 50% to 75%, and this parameter influences the quality of inverse transformation back to audio signal. However, the bigger the overlap parameter, the wider the resulting spectrogram is. Therefore, we have chosen 50% overlap to preserve quality transformation while maintaining a reasonable width of our spectrogram, thus the time it covers.

From the choice of overlap parameter, we know that $H = N/2$ and now we have to decide frame size. Frame size also influences the height of the spectrogram, $height = N/2 + 1$, and it is wise to choose it as a power of two, so we can use Fast Fourier Transform, in short FFT, for faster computation. For a good quality of inverse transformation, frame size 2048 is used in audio engineering. However, that would result in a $height = 1025$, which would be computationally very expensive. Therefore, we chose $N = 512$, resulting in $height = 257$ with our choice of overlap ($H = 256$).

Now we need to decide at what threshold we want to pad or trim our spectrograms, and we chose 256 columns, which is about 1.5 seconds of audio with a 44.1 kHz sample rate and $N = 512$, $H = 256$. As it was with MFCC representation, we have to find the right values to pad the spectrogram with, but with STFT it is easy; we can pad it with 0, because this way we add a few columns without any energy; therefore, it will not make any sound.

Our resulting STFT spectrogram has $shape = (257, 256, 1)$.

6.2.4 VAE_4 experiments

At the end of the subsection 6.2.2 we mentioned a few experiments with VAE_4 that copied the experiments we did with VAE_3 but without significant differences in the quality of the generated results.

The next experiments work with two VAE_4 modifications: VAE_4_stft, VAE_4_2_channels, and both of these modifications are made for STFT representation.

In our first tries with STFT representation, we wanted to predict both magnitude and phase; therefore, we took an STFT spectrogram, split it into two matrices, one for magnitude and the second for angle phase, and concatenated them into a tensor with $shape = (257, 256, 2)$.

For this representation, we used VAE_4_2_channels, a modified VAE_4 for this input shape with a 2000 epoch and a 0.001 learning rate. We tried 4 KL regularisation values: 0.01, 0.001, 0.0001, 10^{-5} , and two noise settings: no noise and multiplicative noise with $mean = 1$ and $std = 0.05$. This model had worse results than models trained on MFCC representation: the higher values of KL regularisation were not able to reconstruct samples because the majority of them got encoded into the same distribution, and for lower KL regularisation, the reconstructions did capture decay and volume, but they contained a new type of

noise artefact that was stronger than the one that occurred in MFCC models.¹²

We first thought that the problem was with high KL regularisation values and short training; therefore, we made a new experiment with even lower KL regularisation values: 10^{-5} , 10^{-6} , 10^{-7} , and 5000 epochs with a learning rate 0.0001. Additionally, we trained these models on a subset of all drum categories: kick, snare, crash, clap, and tom. However, the same noise artefact occurred as in the previous batch.¹³

Lastly, we tested training a model for each drum category to examine whether the new noise artefact problem is not caused by the combining of the multiple categories of samples in one latent space. We trained every model for 3000 epochs with a 0.0001 learning rate, KL regularisation values: 0, 10^{-5} , 10^{-6} , 10^{-7} , and noise settings the same as in the last experiments. Reconstruction got better because the network had to remember only a fraction of the dataset, but these models had comparable results with the models trained on the whole dataset and the noise artefact preserved.¹⁴

We then tried to throw away the reconstructed phase and estimated it with Griffin-Lim algorithm, and the noise artefact changed to the original noise artefact seen in the MFCC models, but it was more subtle. Hence, we ditched the idea of reconstructing the phase with the network and found a new possible origin for the original noise artefact.

In another experiment, we trained the model only on the magnitude of the STFT spectrogram and then recovered the phase with Griffin-Lim algorithm. We trained a new modification of VAE_4, VAE_4_STFT designed for magnitude STFT input, for 5000 epochs with learning rate 0.0001, 4 KL regularisation values: 0, 10^{-5} , 10^{-6} , 10^{-7} and two noise settings: no noise and normal multiplicative noise with $mean = 1$ and $std = 0.5$. This model delivered good results. The first three principal components were changing the decay length, pitch, and volume, and sampling from prior generated some interesting samples. However, the samples were still noisy and contained the same noise artefact as in MFCC representation, even though it was less significant, and the reconstruction of training data was not perfect.¹⁵

Therefore, we decided to create another bigger architecture. Although the noise artefact could be caused by the Griffin-Lim algorithm because both inverse MFCC and our iSTFT transformation use it.

6.2.5 VAE_magpie_1 experiments

We designed the last architecture used in our experiments. Our goal was to bring the reconstruction loss to a minimum to achieve an almost perfect reconstruction. As we described in subsection about this model 6.1.3, this model had residual connections, batchnorms, and more layers with more channels, and

¹²The directory with the results:

“results/VAE_4_2_channels_conversion=stft_epochs=2000”

¹³The directory with the results:

“results/VAE_4_2_channels_conversion=stft_epochs=5000”

¹⁴We did not attach results of these models, because they very similar to previous two experiments.

¹⁵The directory with the results:

“results/all_VAE_4_stft_conversion=stft_gl_epochs=5000”

the whole training time increased multiple times, even though the number of epochs needed to achieve convergence decreased significantly.

We conducted the first experiment with 3000 epochs, a learning rate 0.0001, KL regularisation 0, and multiplicative noise with $mean = 1$ and $std = 0.3$. The model achieves almost ten times lower reconstruction loss than the best model based on VAE_4 architecture. The reconstruction was much clearer and less noisy, and even more unusual samples got reconstructed quite well.¹⁶

Lastly, we conducted a series of experiments with latent loss. We trained models with KL regularisation: 10^{-4} , 10^{-5} , 10^{-6} , 10^{-7} , 10^{-8} ; two noise settings: no noise and multiplicative noise with $mean = 1$ and $variance = 0.3$; for 2500 epochs with a learning rate 0.0001. After the examination, we decided that the best model was the one without the noise and KL regularisation 10^{-6} . However, we encountered a bug in the preprocessing pipeline where we were padding the STFT magnitude with the wrong values, resulting in a weird noise at the end of samples with shorter decays. We fixed it, but we did not have enough time to train a new batch due to the very long training time. The noise artefact was still preserved, but it was much less prominent than in previous experiments. Even though this experiment produced the best results, with solid reconstruction capabilities and fine sampling.¹⁷

Therefore, our hypothesis is that the noise artefact is caused either by the imperfect phase estimation with Griffin-Lim algorithm or by our parameters of the STFT and MFCC representations because they do not provide enough accuracy for better estimation of the phase. We assume this because of the additional roundtrip experiments we conducted focusing on cymbal samples and the reconstruction of the phase with Griffin-Lim algorithm. We found out that the same noise artefact occurred, but it was very subtle, and we had to pay extra attention to the results in order to hear it.

6.3 Failed Experiments

In this last section, we will describe the experiments that were not successful because the training was not stable or the results did not surpass our previous models.

The VAE_2 model that we mentioned previously was a model that had approximately the same layers as VAE_4 model, except the number of channels was four times larger. This model had problems with stable training; usually it diverged or got stuck in a local minimum, and if it did not, the training was really slow. Even with multiple experiments with various KL regularisation values, we could not manage to get rid of these problems. We think these problems were caused by two main factors.

The first was the exploding logvar, which made the model diverge after the first few batches. We had the same problem with VAE_magpie_1 which was an even bigger model than the VAE_2. In smaller models, this problem did not occur, so we think that the first few layers of encoder created values too big before

¹⁶The directory with the results:

“results/all__VAE_magpie_1_kl-reg=0__conversion=stft_gl__epochs=3000”

¹⁷The directory with the results:

“results/all__VAE_magpie_1__conversion=stft_gl__epochs=2500”

the logvar FC layer, and the layer with default initialization did not manage to pull it down enough. Then a chain reaction occurred, and the logvar exploded into values that, after applying the exponential function, became plus or minus infinities. In VAE_magpie_1, we solved this problem by initialising the weights of the FC logvar layer closer to zero, so the backpropagation updates had enough time to reduce the exploding effect.

The second factor was a phenomenon called the vanishing or exploding gradient. This phenomenon occurs when the gradients of the loss function become extensively large or small. This happens in deeper networks because during the backpropagation, the gradients of currently updated layers are multiplied by the gradients from previous layers; therefore, if the gradients are bigger than one, they explode within the first layers, or if they are smaller than one, they vanish. This causes training to diverge if the gradients explode, especially with the logvar FC layer, because the updates performed on the first few layers are too large, and it causes very slow training when the gradient vanishes because updates on the first layers are very small. This problem can be solved by more appropriate initialization of the layers and regularisation methods such as residual connections and batchnorms. We are using all of these methods in VAE_magpie_1; therefore, its training is faster and much more stable even when the model is larger than VAE_2.

In experiments, we also tried to add a dropout layer in our architectures; specifically, we tried it on VAE_3 and later on VAE_4 models and MFCC representation. The dropout is a regularisation method that prevents the overfitting of the model by setting to zero (dropping out) some randomly chosen neurons with each update. This way, we force the neurons to be less reliant on other neurons and force them to create meaningful features on their own.

We tried several experiments with a dropout 0.1, but none of them were successful. Either the encoder encoded all of the samples into one point or the encoder did the job right and the decoder learned to create average noise. We did not manage to find the right value of KL divergence to prevent this from happening, so we decided to focus on different methods.

Another method we tried was the standard scaler, a method where the input is scaled into a tighter range of values. This process is reversible; therefore, there should not be a problem with extracting the original range of values from the network's output. Similarly to dropout, we tried this method on VAE_3 and VAE_4 models and MFCC representation.

However, we ran into the same problems as with the dropout method, and after multiple experiments with various KL regularisation values, we did not manage to achieve meaningful results.

We designed a VAE_5_tied_weights model, which is a model where the encoder and decoder use the same weights for each layer; therefore, they have to be perfectly mirrored. The weights are updated twice in one batch run: the first time in backpropagation of the decoder and the second time in backpropagation of the encoder. This way, we could achieve fewer parameters and faster training.

This approach did not bring any significant results and was a little worse than models without the tied weights, even with a similar number of parameters; therefore, we decided to abandon this plan.

Lastly, we tried an experiment with VAE_4_2_channels, where we trained

the model on both the magnitude and angle phase of the STFT spectrogram, and then during reconstruction of the spectrogram back to the audio signal, we threw away the reconstructed phase and estimated it with Griffin-Lim algorithm. We thought that the presence of the phase during training could improve the learned features of the model. However, the training was longer due to the two-channel input and did not provide better results than training the model only on the magnitude of the STFT spectrogram.

7 Conclusion

In preparation for this thesis, we arranged several meetings with electronic music producers to explore their workflow with the intention of designing a tool based on statistical learning to make some parts of their workflow faster and more interactive. Based on these meetings, we introduced the topic of generating drum samples with statistical learning.

In the research part, we explored various techniques of statistical learning that are used for audio generation and synthesis, investigated their advantages and disadvantages, and selected the Variational Autoencoder, which was best suited for our goal. Additionally, we researched multiple audio representations, reviewed relevant literature on how they perform in generation tasks, tested them in our settings, and picked MFCC and STFT representations as they had the best preconditions to succeed in our work.

In the third chapter, “Methods” 3, we explained the VAE architecture in detail, described the loss function, and demonstrated the importance of regularising the two components of the loss function, reconstruction and latent loss. Furthermore, we mentioned advantages and disadvantages of the VAE architecture, along with motives that convinced us to select this architecture. Secondly, we explained how the audio signal is processed into digital form, along with introducing the 2D time-frequency audio representation called a spectrogram and revealing the differences with raw waveform representations. On top of that, we explained two spectrogram representations we selected earlier for our work, MFCC and STFT, and indicated which parameters of these representations will be crucial for successful results. Finally, we have shown that convolutional and transposed convolutional layers are well suited for processing the spectrogram representation, and we have determined the architectures of the encoder and the decoder.

We introduced our own dataset with almost 16,000 drum samples divided into 9 categories based on the type of drums, that was prepared specifically for this task. We explained what the samples are and how artists can obtain them. In the end, we showed the whole process of finding, collecting, cleaning, inspecting, and organising these samples that was necessary for the creation of this dataset.

We described the problem of evaluating generative models and put emphasis on the importance of a human examiner. Therefore, we created several examination methods consisting of plots showing properties of the latent space and methods of reconstructing train data or sampling new data from the latent space using principles like PCA and convex combinations. Finally, we wrapped these methods into an examination pipeline that generated all of these outputs, ready for human evaluation.

We presented three VAE architecture designs with the encoder and the decoder consisting of convolutional and transposed convolutional layers, and the third was enriched with residual connections and batch normalisations for faster and more stable training. We also described the choice of parameters for both audio spectrogram representations, and based on these parameters, we derived the input shapes.

We conducted several batches of experiments that tested the performance of architectures, audio representations, parameter settings, and other features.

Additionally, we reviewed the results generated by the examination pipeline for every experiment and summarised what the experiment did better than previous ones and what problems arose. In the end, these experiments delivered a model that had solid reconstruction capabilities and could generate new meaningful data, especially with PCA shift and convex combination methods.

Shifting the samples in directions of the first few principal components from the PCA computed from the projection of data into the trained latent space caused interpretable high-level changes to the samples: increasing and decreasing the volume, extending or shortening the decay, or changing the pitch. Additionally, convex combinations of samples from different drum categories generated new sounds that were a combination of them; for example, a combination of kick and clap got decoded into a sound whose hit was strongly affected by the kick, but the decay originated from the clap. This resembles the behaviour of well-trained embeddings.

The latent space was well organised, and the samples that shared the same category were usually encoded close to each other, forming clusters. This has been achieved without using any labels.

Lastly, we did the training on a relatively small dataset in contrast to some other papers with more than a hundred thousand samples, and we note that these usually do not include the slowly decaying cymbal sounds.

7.1 Future work

However, the model still had issues that could be resolved in future work. As we mentioned in the chapter “Experiments and Results” 6, the final model’s results still had the same artefact we were trying to remove, and our hypothesis is that it is either caused by the phase recovery Griffin-Lim algorithm or insufficient precision of the STFT parameters. We found another non-iterative algorithm for transforming STFT magnitude into an audio signal called PGHI, introduced by Holighaus et al. [35]. The authors of this paper claim significantly better performance in contrast to the Griffin-Lim algorithm. The other part of the hypothesis could be solved by increasing the number of FFTs and the overlap parameters of the STFT representation, resulting in improved precision of the inverse STFT transform but also in a larger input shape. To test this change, we would need more computational resources because of the extended capacity and training time of the model.

Another problem we encountered was averaging noise in our samples, which is caused by the typical behaviour of the VAE architecture. To solve this, at least from the user’s perspective, we could apply various noise cancellation software that is available.

After these steps, if we are able to generate new data with sufficient quality, we could develop a programme or a VST plugin¹ that could bring these models to artists for everyday use in electronic music production. Therefore, we could examine these models with the help of professionals, deliver their opinions about the results, and start moving the promising thesis results closer to production.

¹Plugin for professional music production programmes

Bibliography

1. HAWLEY, Scott H; CHATZIANNOS, Vasileios; MORRISON, Andrew. Synthesis of musical instrument sounds: Physics-based modeling or machine learning. *Phys. Today*. 2020, vol. 16, no. 1, pp. 20–28.
2. MEHRI, Soroush; KUMAR, Kundan; GULRAJANI, Ishaan; KUMAR, Rithesh; JAIN, Shubham; SOTELO, Jose; COURVILLE, Aaron; BENGIO, Yoshua. SampleRNN: An unconditional end-to-end neural audio generation model. *arXiv preprint arXiv:1612.07837*. 2016.
3. OORD, Aaron van den; DIELEMAN, Sander; ZEN, Heiga; SIMONYAN, Karen; VINYALS, Oriol; GRAVES, Alex; KALCHBRENNER, Nal; SENIOR, Andrew; KAVUKCUOGLU, Koray. Wavenet: A generative model for raw audio. *arXiv preprint arXiv:1609.03499*. 2016.
4. ENGEL, Jesse; RESNICK, Cinjon; ROBERTS, Adam; DIELEMAN, Sander; NOROUZI, Mohammad; ECK, Douglas; SIMONYAN, Karen. Neural audio synthesis of musical notes with wavenet autoencoders. In: *International Conference on Machine Learning*. PMLR, 2017, pp. 1068–1077.
5. KINGMA, Diederik P; WELLING, Max. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*. 2013.
6. ESLING, Philippe; CHEMLA-ROMEY-SANTOS, Axel; BITTON, Adrien. Generative timbre spaces with variational audio synthesis. In: *Proceedings of the International Conference on Digital Audio Effects (DAFx)*. 2018, pp. 175–181.
7. BITTON, Adrien; ESLING, Philippe; CHEMLA-ROMEY-SANTOS, Axel. Modulated variational auto-encoders for many-to-many musical timbre transfer. *arXiv preprint arXiv:1810.00222*. 2018.
8. AOUMEUR, Cyran; ESLING, Philippe; HADJERES, Gaëtan. Neural drum machine: An interactive system for real-time synthesis of drum sounds. *arXiv preprint arXiv:1907.02637*. 2019.
9. TOLSTIKHIN, Ilya; BOUSQUET, Olivier; GELLY, Sylvain; SCHOELKOPF, Bernhard. Wasserstein auto-encoders. *arXiv preprint arXiv:1711.01558*. 2017.
10. GOODFELLOW, Ian; POUGET-ABADIE, Jean; MIRZA, Mehdi; XU, Bing; WARDE-FARLEY, David; OZAIR, Sherjil; COURVILLE, Aaron; BENGIO, Yoshua. Generative adversarial networks. *Communications of the ACM*. 2020, vol. 63, no. 11, pp. 139–144.
11. DONAHUE, Chris; MCAULEY, Julian; PUCKETTE, Miller. Adversarial audio synthesis. *arXiv preprint arXiv:1802.04208*. 2018.
12. GRIFFIN, Daniel; LIM, Jae. Signal estimation from modified short-time Fourier transform. *IEEE Transactions on acoustics, speech, and signal processing*. 1984, vol. 32, no. 2, pp. 236–243.
13. MARAFIOTI, Andrés; PERRAUDIN, Nathanaël; HOLIGHAUS, Nicki; MAJDAK, Piotr. Adversarial generation of time-frequency features with application in audio synthesis. In: *International conference on machine learning*. PMLR, 2019, pp. 4352–4362.

14. ENGEL, Jesse; AGRAWAL, Kumar Krishna; CHEN, Shuo; GULRAJANI, Ishaan; DONAHUE, Chris; ROBERTS, Adam. Gansynth: Adversarial neural audio synthesis. *arXiv preprint arXiv:1902.08710*. 2019.
15. NISTAL, Javier; LATNER, Stefan; RICHARD, Gael. Drumgan: Synthesis of drum sounds with timbral feature conditioning using generative adversarial networks. *arXiv preprint arXiv:2008.12073*. 2020.
16. DRYSDALE, Jake; TOMCZAK, Maciej; HOCKMAN, Jason. Adversarial synthesis of drum sounds. In: *Proceedings of the 23rd International Conference on Digital Audio Effects (DAFx2020)*. 2020, pp. 167–172.
17. DRYSDALE, Jake; TOMCZAK, Maciej; HOCKMAN, Jason. Style-based drum synthesis with GAN inversion. In: *Extended Abstracts for the Late-Breaking Demo Session of the 22nd Int. Society for Music Information Retrieval Conference*. International Society for Music Information Retrieval, 2021.
18. LAVAUULT, Antoine; ROEBEL, Axel; VOIRY, Matthieu. StyleWaveGAN: Style-based synthesis of drum sounds with extensive controls using generative adversarial networks. *arXiv preprint arXiv:2204.00907*. 2022.
19. ADLER, Jonas; LUNZ, Sebastian. Banach wasserstein gan. *Advances in neural information processing systems*. 2018, vol. 31.
20. KARRAS, Tero; LAINE, Samuli; AILA, Timo. A style-based generator architecture for generative adversarial networks. In: *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 2019, pp. 4401–4410.
21. RAMIRES, António; CHANDNA, Pritish; FAVORY, Xavier; GÓMEZ, Emilia; SERRA, Xavier. Neural percussive synthesis parameterised by high-level timbral features. In: *ICASSP 2020-2020 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*. IEEE, 2020, pp. 786–790.
22. STOLLER, Daniel; EWERT, Sebastian; DIXON, Simon. Wave-u-net: A multi-scale neural network for end-to-end audio source separation. *arXiv preprint arXiv:1806.03185*. 2018.
23. RONNEBERGER, Olaf; FISCHER, Philipp; BROX, Thomas. U-net: Convolutional networks for biomedical image segmentation. In: *Medical image computing and computer-assisted intervention—MICCAI 2015: 18th international conference, Munich, Germany, October 5-9, 2015, proceedings, part III 18*. Springer, 2015, pp. 234–241.
24. ALAIN, Guillaume; CHEVALIER-BOISVERT, Maxime; OSTERRATH, Frederic; PICHE-TAILLEFER, Remi. Deepdrummer: Generating drum loops using deep learning and a human in the loop. *arXiv preprint arXiv:2008.04391*. 2020.
25. NISTAL, Javier; LATNER, Stefan; RICHARD, Gaël. Comparing representations for audio synthesis using generative adversarial networks. In: *2020 28th European Signal Processing Conference (EUSIPCO)*. IEEE, 2021, pp. 161–165.
26. ALLEN, J. Short term spectral analysis, synthesis, and modification by discrete Fourier transform. *IEEE Transactions on Acoustics, Speech, and Signal Processing*. 1977, vol. 25, no. 3, pp. 235–238. Available from DOI: 10.1109/TASSP.1977.1162950.

27. IMAI, Satoshi. Cepstral analysis synthesis on the mel frequency scale. In: *ICASSP'83. IEEE International Conference on Acoustics, Speech, and Signal Processing*. IEEE, 1983, vol. 8, pp. 93–96.
28. LE ROUX, Jonathan; ONO, Nobutaka; SAGAYAMA, Shigeki. Explicit consistency constraints for STFT spectrograms and their application to phase reconstruction. In: *SAPA@ INTERSPEECH*. Citeseer, 2008, pp. 23–28.
29. MCFEE, Brian; RAFFEL, Colin; LIANG, Dawen; ELLIS, Daniel PW; MCVICAR, Matt; BATTENBERG, Eric; NIETO, Oriol. librosa: Audio and music signal analysis in python. In: *SciPy*. 2015, pp. 18–24.
30. HE, Kaiming; ZHANG, Xiangyu; REN, Shaoqing; SUN, Jian. Deep residual learning for image recognition. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
31. PASZKE, Adam; GROSS, Sam; MASSA, Francisco; LERER, Adam; BRADBURY, James; CHANAN, Gregory; KILLEEN, Trevor; LIN, Zeming; GIMELSHEIN, Natalia; ANTIGA, Luca, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*. 2019, vol. 32.
32. WOLD, Svante; ESBENSEN, Kim; GELADI, Paul. Principal component analysis. *Chemometrics and intelligent laboratory systems*. 1987, vol. 2, no. 1-3, pp. 37–52.
33. KILGOUR, Kevin; ZULUAGA, Mauricio; ROBLEK, Dominik; SHARIFI, Matthew. Fr'echet audio distance: A metric for evaluating music enhancement algorithms. *arXiv preprint arXiv:1812.08466*. 2018.
34. HE, Kaiming; ZHANG, Xiangyu; REN, Shaoqing; SUN, Jian. Delving deep into rectifiers: Surpassing human-level performance on imagenet classification. In: *Proceedings of the IEEE international conference on computer vision*. 2015, pp. 1026–1034.
35. HOLIGHAUS, Nicki; KOLIANDER, Günther; ABREU, Luis Daniel; PRUŠA, Zdenek. Non-iterative phaseless reconstruction from wavelet transform magnitude. In: *Proc. 22nd Int. Conf. Digit. Audio Effects*. 2019, pp. 2–6.

List of Figures

3.1	autoencoder	16
3.2	Issue with autoencoders generating new data.	16
3.3	Latent space of autoencoder and Variational autoencoder	17
3.4	Variational autoencoder	19
3.5	Latent spaces trained with various regularisations of loss	20
3.6	Audio wave of a crash one-shot	22
3.7	Spectrogram of a kick one-shot	23
3.8	Hann window applied to a frame	24
3.9	MFCC spectrogram	26
3.10	MFCC pipeline	27
3.11	Griffin-Lim algorithm reconstruction after 0, 5, 10, and 15 iterations. Note that we used 32 iterations in our experiments, but we illustrated only iterations up to 15, because more iterations do not make a difference in the illustrations, but in the audio signal, they do.	28
4.1	Table of counts for drum categories and genres categories.	31
4.2	Diagram of steps used in data cleaning	33
6.1	VAE_3 network architecture	42
6.2	VAE_4 network architecture	43
6.3	Residual convolution and deconvolution blocks used in VAE_magpie_1.	44
6.4	VAE_magpie_1 network architecture	45

List of Abbreviations

- AE: autoencoder
- VAE: variational autoencoder
- STFT: short-time fourier transform
- MFCC: Mel-frequency cepstral coefficient
- PCA: principal component analysis

A Attachments

A.1 Installation instructions

We could not upload the attachment with results and dataset to the “Information System” due to problem with too many files in the attachment, therefore we uploaded them into the google drive and they are available on this url: <https://drive.google.com/file/d/1kEZowmThN2kIpij7qlYHgufzb3iGGY8-/view?usp=sharing>.

1. Create a new conda environment:

```
“conda create -n myenv -c conda-forge python=3.11”
```

2. Activate conda environment:

```
“conda activate myenv”
```

3. Install the packages

```
“conda install -c conda-forge torch librosa matplotlib scikit-learn pysoundfile  
scipy numpy python-sounddevice”
```

The training script and evaluation script directories:

- Training script: “src/VAE/training/train.py”
- Evaluation script: “src/VAE/evaluation/eval.py”

All of the arguments for these scripts are documented in the scripts themselves.

There are also scripts to start training on metacentrum in src/metacentrum_training_scripts.